

AGENTIC DESIGN

The Agentic Designer

How AI Agents, Claude Design, Open Design, and the New Design Tool Ecosystem Are Transforming Product Design

Approach: Agentic Design Workflows

Version: 0.1.0 · **Date:** May 2026

Mehran Mozaffari

github.com/imehr

This book was written with the assistance of AI tools. All tool references and code examples reflect capabilities as of May 2026. The agentic design tool ecosystem evolves rapidly; check official documentation for the latest features and integrations.

The Agentic Designer

How AI Agents, Claude Design, Open Design, and the New Design Tool Ecosystem Are Transforming Product Design

Copyright © 2026 Mehran Mozaffari. All rights reserved.

First Edition: May 2026

Personal Use License

This book is free for personal, non-commercial use. You may read, share, and reference it for individual study, education, and research.

No part of this publication may be sold, rented, or redistributed for commercial purposes without written permission from the author.

For permissions and licensing inquiries:

<https://github.com/imehr>

Written with AI Assistance

This book was written with the assistance of AI tools. The author directed, reviewed, and approved all content. AI tools were used for drafting, research, and editing. All opinions and analyses are the author's own.

All tool references and code examples reflect capabilities as of May 2026. The agentic design tool ecosystem evolves rapidly; check official documentation for the latest features and integrations.

Trademarks

Product names, logos, and brands mentioned in this book are the property of their respective owners. Use in this book is for identification and educational purposes only and does not imply endorsement.

Claude Code and Claude Design are products of Anthropic. OpenAI, Codex, and ChatGPT are products of OpenAI. Gemini is a product of Google. Figma is a product of Figma, Inc. Miro is a product of Miro. GitHub Copilot is a product of GitHub, Inc. Vercel and v0 are products of Vercel, Inc. Bolt.new and WebContainers are products of StackBlitz. Lovable is a product of Lovable AB. Remotion is a product of Remotion GmbH. shadcn/ui is a product of shadcn. Supabase is a product of Supabase Inc. tldraw is a product of tldraw GmbH. Paper is a product of Paper Design Inc. Pencil is a product of Pencil.dev. Kiro is a product of Amazon Web Services. Hyperframes is a product of HeyGen. OpenCode is a product of OpenCode AI. ElevenLabs is a product of ElevenLabs Inc. Playwright is a project of the Microsoft team.

Disclaimer

This book is provided for informational and educational purposes only. Nothing in this book constitutes legal, financial, tax, or professional advice. The author makes no warranties about the accuracy or completeness of the information.

Tool capabilities, pricing, API availability, and feature sets described in this book may have changed since publication. Always consult the official documentation for each tool or platform for current information.

Code examples are provided for educational purposes and should be tested in your own environment before use in production.

Edition and Publisher Details

Published by the author
Sydney, Australia

Edition: First Edition, May 2026
Version: 1.0

Credits

Author: Mehran Mozaffari

Editorial review: Multi-model AI review pipeline

Technical reviewers: Claude Opus 4.6, Gemini 3.1 Pro

Design and production: Agentic publishing pipeline (OpenCode)

Open Source Acknowledgments

This book references and describes the following open-source and source-available projects. The author gratefully acknowledges the contributions of their maintainers and communities:

- Open Design (nexu-io) — MIT License
- OpenPencil (ZSeven-W) — MIT License
- Huashu Design (alchaincyf) — MIT License
- Remotion (remotion-dev) — Remotion License
- tldraw — Apache License 2.0
- ComfyUI — GPL-3.0 License
- CLI-Anything (HKUDS) — MIT License
- OpenCode (anomalyco) — MIT License

This book does not distribute source code from any of these projects. References are descriptive and educational.

Contact the Author

Blog: <https://piazzr.github.io/applied-ai/>

GitHub: <https://github.com/imehr>

For corrections, errata, or licensing inquiries, please open an issue on the book's repository or contact the author through the channels above.

Contents

The Agentic Designer — 14 chapters + 3 appendices

Part I: Foundations

01 The Agentic Design Paradigm

02 Your Agent Toolkit

03 Teaching Agents to Design

04 Design-as-Code

Part II: Design Tools

05 Paper and Pencil

06 Open Design and OpenPencil

07 Huashu Design

08 Design Systems and Tokens

Part III: Advanced Workflows

09 Motion and Video

10 MCP Integrations

11 Multi-Agent Design Teams

12 Production UI from Design

Part IV: Practice and Future

13 Real-World Case Studies

14 The Future of Agentic Design

15 Agentic Presentations

16 Building Agent Pipelines for Design

17 DESIGN.md: Portable Design Systems

Appendices

A Tool Comparison Matrix

B MCP Server Reference

C Prompt Library for Design Tasks

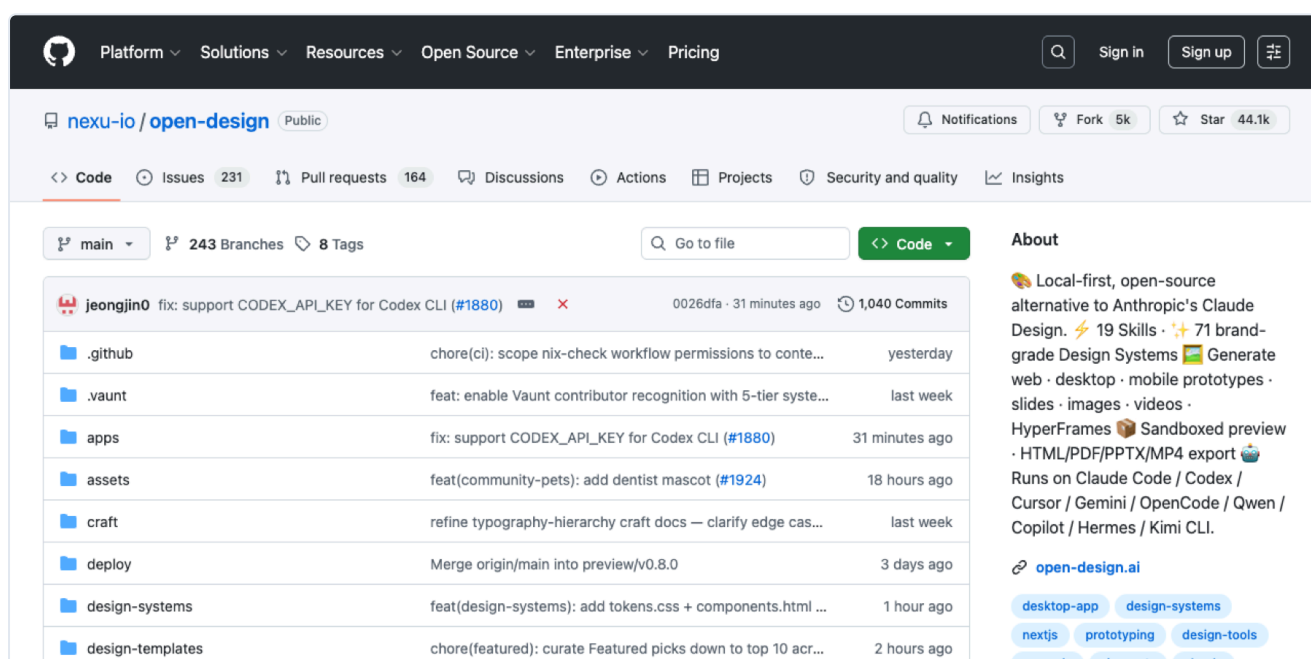
01 The Agentic Design Paradigm

Why AI agents are becoming the new design tool interface

The way software gets designed is changing. AI coding agents --- tools built to write and ship code --- are now operating design tools, generating visual artifacts, and closing the loop from design intent to production output. This chapter explains why that shift happened, what enables it, and what it means for anyone who designs software for a living.

The Shift from GUI to Agent

For a decade, the design workflow looked the same. Open Figma or Sketch. Use a mouse-driven GUI to place elements on a canvas. Export static images or specs. Hand off to engineering. Engineering interprets the design, writes code, and ships something that approximates the original intent.



The four major agent platforms compared by maker, core capability, and key differentiator

That loop worked. It still works. But it has a structural problem: the design artifact and the production artifact are two different things. A Figma file is not a website. A Sketch mockup is not a component library. Every handoff introduces translation loss. The designer describes a hover state. The engineer implements a different one. The designer specifies a 12px gap. The engineer uses 16px because it looked better in code. Small divergences accumulate into a final product that only loosely resembles the original design.

AI coding agents change this equation. An agent can read a design file, generate code from it, write HTML back to a canvas, and iterate on visual output --- all without leaving the terminal. The design artifact and the production artifact converge toward the same thing.

Consider what the four major agent platforms can do as of May 2026:

Platform	Maker	Core capability	Key differentiator
Claude Code	Anthropic	Agentic coding across terminal, IDE, desktop, web	Richest skill system, most mature MCP support
Codex CLI	OpenAI	Cloud-based agent running tasks in isolated sandboxes	Parallel task execution, citation-backed output
OpenCode	Open source	Multi-provider agent with 75+ LLM options	160K GitHub stars, cross-compatible with Claude Code conventions
Gemini CLI	Google	Terminal agent with 1M-token context window	Free tier with 6K code requests/day

These tools were built for software engineering. But their capabilities --- reading files, editing code, running commands, connecting to external tools --- map directly onto design tasks. The distinction between "coding agent" and "design agent" is eroding fast.

Claude Code describes itself as "an agentic coding tool that reads your codebase, edits files, runs commands, and integrates with your development tools" (source: [Claude Code Overview](#), retrieved 2026-05-18). The critical word is *agentic*. It does not suggest. It does. It reads a design file, generates a visual artifact, and writes it to a canvas --- as an action, not a recommendation.

OpenAI frames the same shift differently: "We imagine a future where developers drive the work they want to own and delegate the rest to agents" (source: [Introducing Codex](#), retrieved 2026-05-18). Replace "developers" with "designers" and the sentence holds. The agent handles the translation between intent and artifact.

OpenCode, the open-source entry with 160K GitHub stars and 7.5M monthly developers (source: [OpenCode](#), retrieved 2026-05-18), takes the broadest approach. It supports 75+ LLM providers, making it accessible regardless of which model you prefer. Its cross-compatibility with Claude Code conventions means design skills transfer between platforms without modification. Gemini CLI rounds out the field with a 1M-token context window and a free tier that handles 6,000 code requests per day --- generous enough for design iteration.

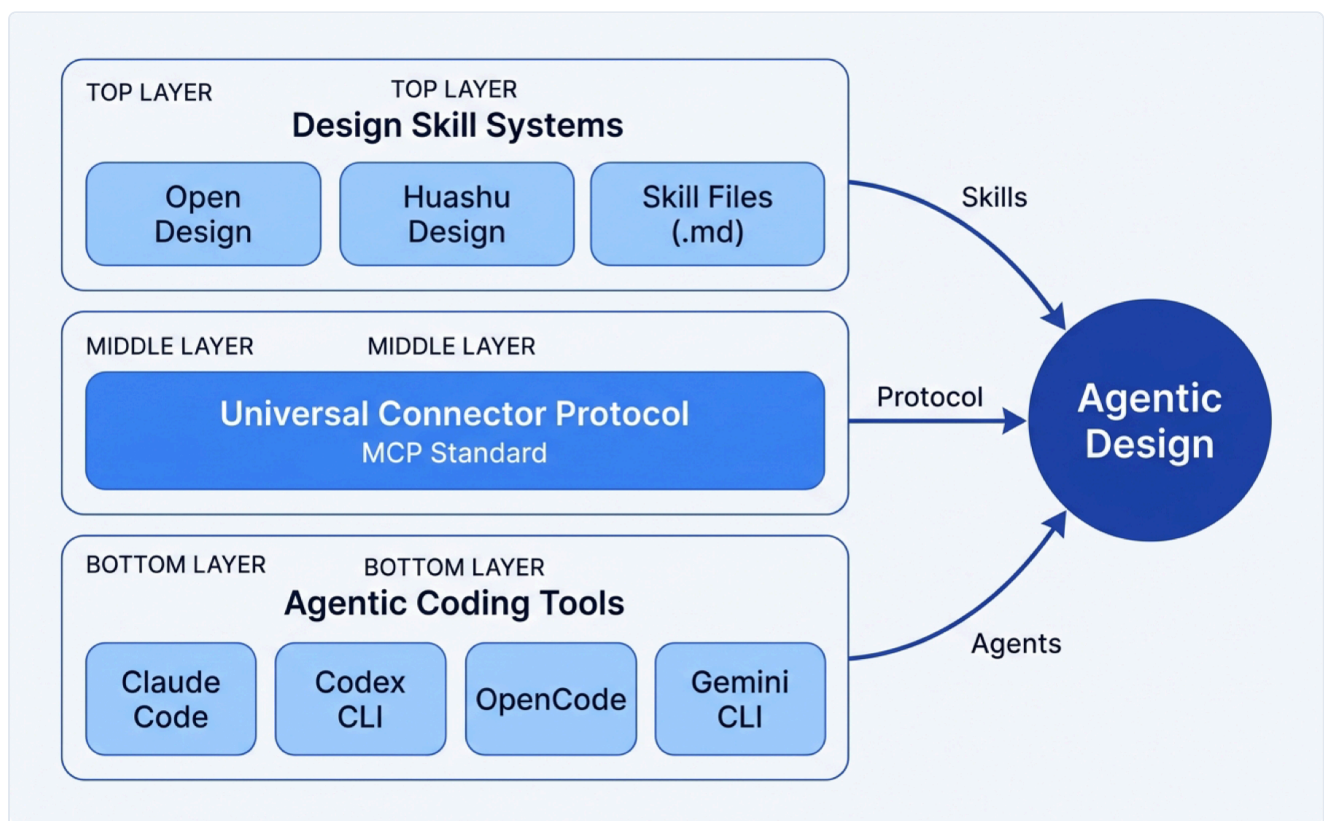
The shift from GUI to agent is not about replacing visual tools. It is about adding a new interface layer. The canvas still matters. The visual preview still matters. What changes is how you operate the canvas. Instead of clicking and dragging, you describe what you want in natural language. The agent translates that description into visual output on the canvas. You review. You iterate. The loop tightens.

This does not mean the mouse-driven GUI disappears. Figma will continue to exist. Sketch will continue to exist. But for a growing class of design tasks --- generating prototypes, creating landing pages, building component libraries, producing motion graphics --- the agent-first workflow is faster, more consistent, and more tightly integrated with production code. The GUI becomes the review surface, not the authoring surface.

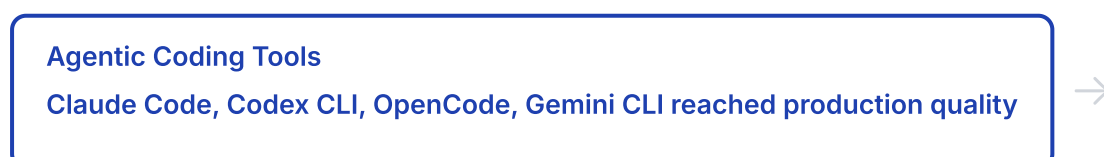
The economics favor the shift. A design engineer with a well-configured agent can produce in one session what used to take a designer and a front-end developer several days of back-and-forth. The agent handles the translation between design intent and code. The human provides direction and quality control. The bottleneck moves from production to decision-making --- which is where it should be.

Why Now: The Convergence of Agents, Skills, and MCP

Three independent developments converged in 2025-2026 to make agentic design possible. None of them is sufficient on its own. Together, they create something new.



Three independent developments converging to enable agentic design in 2025-2026





Agentic coding tools reached production quality. Claude Code matured through 2025, adding skill systems, agent teams, and MCP support. Codex CLI launched with isolated cloud sandboxes and parallel task execution. OpenCode crossed 160K GitHub stars with 900 contributors and 7.5M monthly developers. Gemini CLI entered the field with a generous free tier and 1M-token context window. For the first time, designers had multiple production-grade agents to choose from.

The key milestone was the transition from suggestion-based tools to action-based tools. GitHub Copilot suggests code completions. Cursor suggests edits. These are useful. But they require the designer to drive every action. An agentic tool takes a goal and executes the steps to achieve it. Claude Code reads a codebase, plans an approach, edits multiple files, runs tests, and verifies the result --- all autonomously. That shift from suggestion to action is what enables design workflows.

MCP (Model Context Protocol) became the universal connector. Claude Code documentation describes MCP as "an open standard for connecting AI tools to external data sources" (source: [Claude Code Overview](#), retrieved 2026-05-18). MCP lets an agent connect to Figma, Paper, Pencil, Miro --- any tool that exposes an MCP server. The agent reads designs, exports code, and writes artifacts back to the canvas. All through a standard protocol.

Before MCP, every agent needed custom integrations for every tool. Claude Code would need a Figma integration, a Paper integration, a Pencil integration. Each built differently. Each maintained separately. MCP turns that $N \times M$ problem into an $N + M$ problem. Any agent can connect to any tool that exposes an MCP server. This standardization is what makes multi-tool design workflows practical.

Design-as-code formats emerged. Pencil's .pen files represent vector designs as JSON --- diffable in git, readable by agents, editable in an IDE. OpenPencil's .op files follow the same principle. Design tokens expressed as CSS custom properties or JSON variables become code artifacts that agents can read, transform, and write. The design file is no longer a binary blob. It is a text file the agent can understand, modify, and version.

```
# MCP configuration for a design tool (OpenCode example)
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "my-design-tool": {
      "type": "local",
      "command": ["npx", "-y", "@my-design-mcp/server"],
      "enabled": true,
      "environment": {
        "DESIGN_API_KEY": "your-key"
      }
    }
  }
}
```

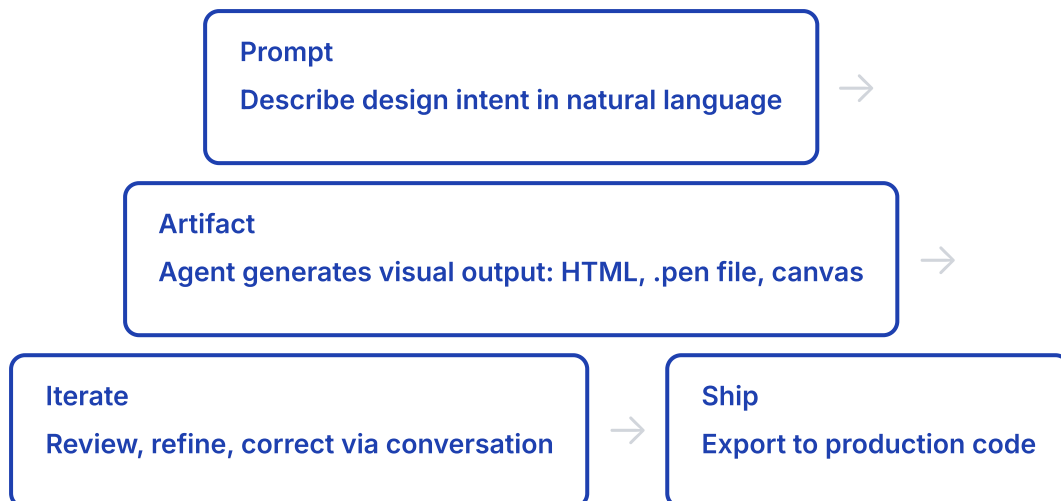
This snippet shows how an MCP server gets configured. The agent connects to the design tool at session start. Every tool operation --- reading designs, writing artifacts, exporting code --- flows through this standard interface.

My take: MCP is the most under-hyped development in this space. Skills and agents get the attention, but MCP is what makes the whole thing work. Without a standard protocol for connecting to design tools, every agent would need custom integrations for every tool. MCP turns that $N \times M$ problem into an $N + M$ problem. Any agent can connect to any tool that exposes an MCP server. This is why I dedicate an entire chapter to MCP integrations later (see [section 10.1](#)).

There is a fourth factor: skill system standardization. Claude Code uses SKILL.md files in `.claude/skills/`. OpenCode uses SKILL.md in `.opencode/skills/`, `.claude/skills/`, or `.agents/skills/`. Both follow the Agent Skills open standard at agentskills.io. This means a design skill written for Claude Code works in OpenCode with zero modification. Skills are portable across agent platforms. I cover this in depth in [section 03.3](#).

The Agentic Design Loop: Prompt → Artifact → Iterate → Ship

Every agent platform implements the same fundamental loop. The details differ, but the pattern is consistent across Claude Code, Codex CLI, OpenCode, and Gemini CLI.



In Claude Code, the loop works like this: describe what you want. The agent reads the codebase, picks a design system from the skill configuration, and generates a visual artifact. You review it --- via screenshot, browser preview, or canvas view. You iterate with natural language corrections. The agent updates the artifact. When the output matches the intent, you export to production code.

Codex runs the same loop in a cloud sandbox. Each task runs for 1-30 minutes, depending on complexity (source: [Introducing Codex](#), retrieved 2026-05-18). The agent provides "verifiable evidence of its actions through citations of terminal logs and test outputs." You review the citations, approve or revise, and merge.

OpenCode splits the loop into two modes. Press Tab to enter Plan mode --- the agent analyzes the request and proposes an approach without making changes. Switch back to Build mode and the agent implements the plan. Use `/undo` to revert bad iterations (source: [OpenCode Docs](#), retrieved 2026-05-18). This two-phase approach reduces wasted work on design tasks where the direction is uncertain.

The design-specific variant of this loop adds a critical layer: the agent reads design system tokens and brand guidelines before generating output. This produces more consistent results and reduces the iteration cycle dramatically. A bare agent might need 5-8 iterations to converge on brand-consistent output. A configured agent --- one that has read your design tokens, color palette, and typography rules --- often gets it right in 1-2 iterations. More on this in [section 03.5](#).

Agent	Prompt method	Iteration	Review
Claude Code	Interactive chat or pipe (<code>c\laude -p</code>)	Conversational refinement	Screenshot, browser, MCP canvas
Codex CLI	Single prompt, sandbox execution	Revise prompt, re-run	Citations: terminal logs, test output
OpenCode	Plan mode (Tab) then Build mode	<code>/undo</code> , <code>/redo</code> , conversational	Screenshot, share link (<code>/share</code>)
Gemini CLI	Terminal prompt	Conversational refinement	Human-in-the-Loop (HiTL) oversight

Notice the pattern. Every agent starts with a prompt. Every agent produces an artifact. Every agent supports iteration. The differences are in how each step works, not in whether the step exists. This consistency is what makes it possible to learn one agent and transfer that knowledge to another.

My take: The Prompt → Artifact → Iterate → Ship loop sounds simple. In practice, the iteration phase is where most of the time goes. An agent that generates a perfect artifact on the first try does not exist. The quality of the output depends on how well you configure the agent's context -- the design system, the brand guidelines, the skill files. Chapters 03 and 08 cover this in detail. Skip those and you will spend your iteration budget on problems the agent could have avoided.

Who This Book Is For and What You Will Learn

This book is for product designers, design engineers, and creative technologists who want to use AI agents as design tools. Not as novelties. Not as experiments. As production tools that ship real work.

The prerequisites are modest. You should be comfortable in a terminal --- navigating directories, running commands, piping output. You should know HTML and CSS fundamentals. You should have used at least one design tool, whether Figma, Sketch, or something else. You do not need to be a software engineer. You do not need to know React deeply, though basic familiarity with JSX concepts helps in later chapters.

What you do not need: a computer science degree, experience with machine learning, or familiarity with prompt engineering as a discipline. The agents handle the technical complexity. Your job is to provide design direction and quality standards.

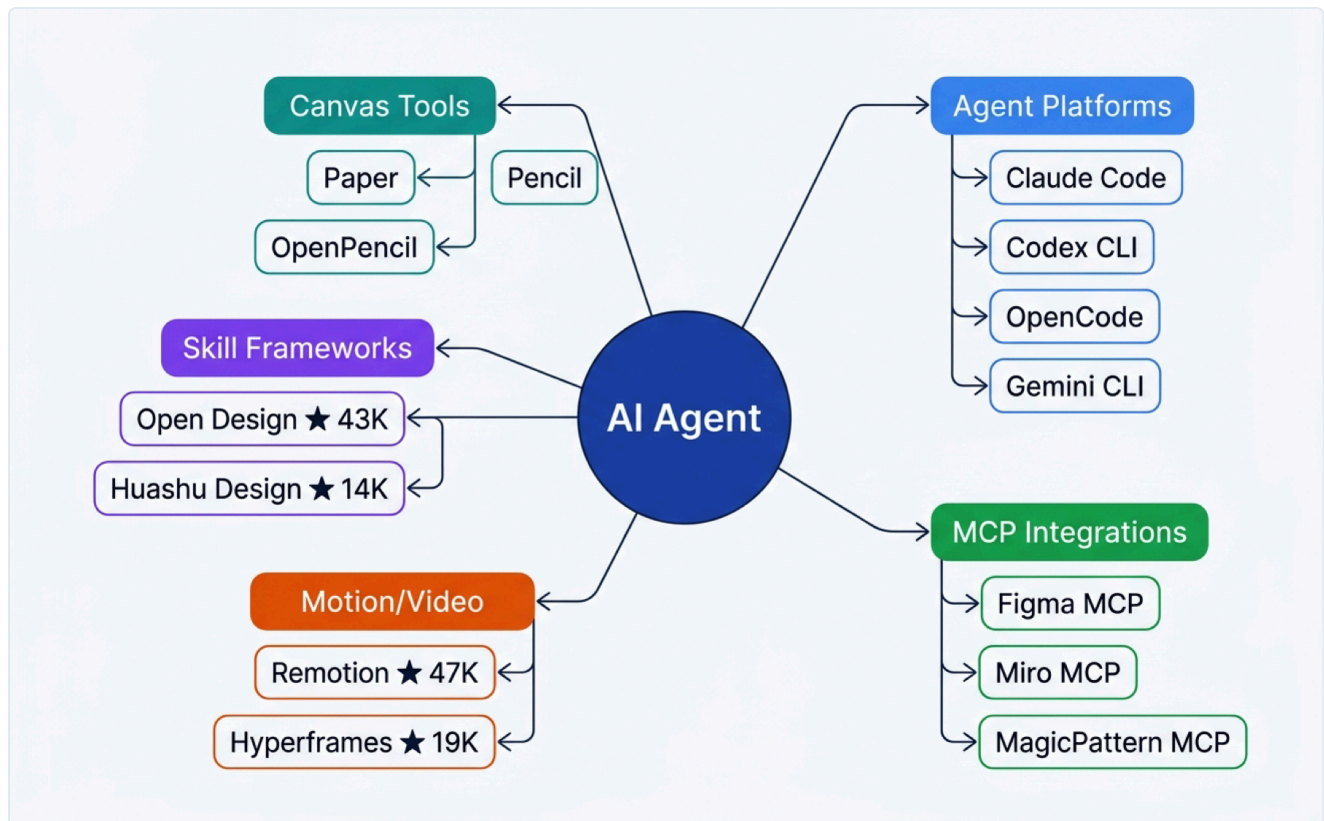
By the end of this book, you will know how to:

- Install and configure the four major AI coding agent platforms.
- Write design instructions in CLAUDE.md and AGENTS.md.
- Install, compose, and create design skills using the SKILL.md convention.
- Connect design tools (Figma, Paper, Pencil, Miro) via MCP servers.
- Use connected canvas tools for agent-driven design workflows.
- Build custom design harnesses that enforce brand consistency.
- Generate production-ready UI code from design artifacts.
- Orchestrate multiple agents for parallel design tasks.
- Create programmatic video and motion graphics with agents.
- Maintain design systems across code and design surfaces.

The book assumes you have access to at least one agent platform. If cost is a concern, OpenCode with a free API key or Gemini CLI's free tier will work for most exercises. The skills and techniques transfer across platforms.

A Map of the Agentic Design Ecosystem

The agentic design ecosystem has five layers. Each layer builds on the one below it. Understanding the layers is more important than memorizing any individual tool.



The agentic design ecosystem organized by tool category and function

Layer	Components	Examples	Covered in
Agent platforms	CLI and IDE agents that execute tasks	Claude Code, Codex CLI, OpenCode, Gemini CLI	Chapter 02
Configuration and skills	Instructions, skills, harnesses	CLAUDE.md, AGENTS.md, SKILL.md, design harnesses	Chapter 03
Design canvas tools	Visual surfaces agents can read and write	Paper, Pencil, OpenPencil	Chapters 05-06
Design skill frameworks	Composable skills for specific design outputs	Open Design, Huashu Design	Chapters 06-07
MCP integrations	Protocol connectors to external design tools	Figma MCP, Miro MCP, MagicPattern MCP	Chapter 10

This book proceeds through these layers from bottom to top. Chapters 02-03 cover agent platforms and configuration. Chapters 04-07 cover design-as-code concepts, canvas tools, and skill frameworks. Chapters 08-10 cover design systems, motion, and MCP integrations. Chapters 11-12 cover multi-agent patterns and the production pipeline. Chapters 13-14 close with case studies and predictions.

A sixth layer exists below these five: the design-as-code formats themselves. .pen files, .op files, HTML as a design surface, design tokens as code variables. These formats are what make the entire stack possible. Without formats that agents can read, write, and diff, none of the layers above would function. I cover this foundational layer in [Chapter 04](#).

The ecosystem also includes motion and video tools. Remotion (47.1k+ GitHub stars) makes videos programmatically with React. Hyperframes (19k+ stars, by HeyGen) uses HTML + GSAP as the authoring format, built specifically for AI agents. These tools extend the agentic design loop beyond static artifacts into time-based media. Chapter 09 covers both in depth.

Tool	Type	Scale (May 2026)	Key format
Open Design	Design skill framework	43K+ GitHub stars, 31 skills, 129 design systems	DESIGN.md, SKILL.md
Huashu Design	HTML-native design skill	14K+ stars, 20 design philosophies	HTML, SKILL.md
Pencil	Vector design tool (IDE)	Commercial, .pen format	.pen (JSON)
OpenPencil	Open-source vector design	3K+ stars, concurrent Agent Teams	.op (JSON)
Remotion	Programmatic video (React)	47.1k+ stars	React/TSX compositions
Hyperframes	Programmatic video (HTML+GSAP)	19k+ stars, by HeyGen	HTML, GSAP

My take: Most people entering this space focus on the agent platform layer. That is the wrong place to start. The agent is a commodity --- Claude Code and OpenCode and Gemini CLI all do roughly the same things. The differentiator is the configuration and skills layer. A well-configured agent with a good skill file will outperform a bare agent every time. Invest your time in chapters 03 and 08 before you invest in learning every agent platform's quirks.

The ecosystem is young and evolving fast. New tools appear monthly. Existing tools add features weekly. The specific tools and numbers in this chapter will age. The mental model --- agents, configuration, canvas, skills, MCP --- will hold. Understanding the layers is more durable than memorizing any tool's feature list.

One more thing worth noting: this ecosystem rewards generalists. The tools are too new for deep specialization. A designer who can configure an agent, write a skill file, and connect an MCP server will outperform a designer who only knows one of those things. This book aims to make you that generalist.

Tip

Next: Chapter 02 compares the four major agent platforms in detail. You will learn how to install each one, what makes it different, and which one to pick for design workflows.

02 Your Agent Toolkit

Claude Code, Codex CLI, OpenCode, and Gemini CLI compared

Four agent platforms dominate the landscape as of May 2026. This chapter covers each one in detail --- installation, authentication, model access, MCP support, skill systems, and CLI ergonomics --- then closes with a decision framework for choosing the right agent for design work.

Agent Platform Landscape

The market settled on four serious contenders by early 2026. Each takes a different approach to the same problem: how to let AI act on your codebase autonomously, accurately, and repeatably.

Feature	Claude Code	Codex CLI	OpenCode	Gemini CLI
License	Proprietary	Open source	Open source	Open source
Models	Claude only	o3, o4-mini	75+ providers	Gemini only
Config file	CLAUDE.md	AGENTS.md	AGENTS.md (CLAUDE.md fallback)	None documented
Skill system	SKILL.md	No	SKILL.md	No
MCP support	Full	Limited (CLI)	Full (local + remote + OAuth)	Yes
Subagents	Yes	No	Yes (General, Explore, Scout)	No
Plan mode	Yes	No	Yes (Tab key)	No
Free tier	No (requires subscription)	API credits	Yes (BYO key)	Yes (1K req/day, as of May 2026)

All data in this table is current as of May 2026. *Pricing and plan details are current as of May 2026 and may change. Check each platform's website for the latest information.* The ecosystem moves fast --- verify before making purchasing decisions. Appendix A contains an extended comparison matrix with pricing details.

Three of the four agents are open source. Only Claude Code is proprietary. This has practical implications: open-source agents can be self-hosted, audited, and modified. Claude Code runs on Anthropic's infrastructure and requires their subscription. For individual designers, this distinction matters less than it does for enterprise teams with compliance requirements.

The model access question is central. Claude Code locks you into Claude models. Gemini CLI locks you into Gemini models. Codex CLI locks you into OpenAI models. OpenCode is the only agent that lets you switch between all of them --- Claude, Gemini, GPT, Llama, Mistral, and dozens more. If you have strong opinions about which model produces the best design output, OpenCode accommodates that preference. If you want to benchmark models against each other on your specific design tasks, OpenCode makes that easy.

That said, model quality for design tasks is converging. As of May 2026, Claude Sonnet and GPT-4-class models produce comparable design output when given equivalent configuration. The differentiator is the configuration layer, not the model. A well-configured agent using a cheaper model will outperform a poorly configured agent using the most expensive model. Chapter 03 covers configuration in detail.

Claude Code (Anthropic)

Claude Code is Anthropic's agentic coding tool. It reads codebases, edits files, runs commands, and connects to external tools via MCP. Available in terminal, VS Code, JetBrains, desktop app, web, and browser (source: [Claude Code Overview](#), retrieved 2026-05-18).

```

claude -- ~/projects/design-hero

> Read the Paper design file and generate a React hero component using MCP

Claude I'll read your Paper design and generate a React component. Let me start by examining the design file.

• pencil_batch_get ("design-hero.pen", patterns: [{name: "HeroSection"}])
  Reading design file at design-hero.pen...
  ✓ Found frame HeroSection (960*540) with 4 children

• pencil_batch_get ("design-hero.pen", nodeIds: ["aK3f2","bN7x1"], readDepth: 3)
  Fetching node details and children...

• pencil_get_variables ("design-hero.pen")
  ✓ Extracted tokens: 12 colors, 8 spacing values, 3 font sizes

I can see the HeroSection frame contains:
— BackgroundImage (frame, 960*540, fill: image)
— GradientOverlay (rectangle, 960*540, fill: linear-gradient 180deg)
— HeadlineText (text, "Design without limits", 48px bold)
— CTAButton (frame, 160*48, rounded: 8px, fill: primary)

Now generating the React component with design tokens...

• Edit src/components/HeroSection.tsx
  ✓ Created src/components/HeroSection.tsx (94 lines)
  Component uses design tokens from variables.css
  All breakpoints match Paper layout constraints

Claude Done. I've created src/components/HeroSection.tsx which maps directly to your Paper design. The component preserves the 960*540 layout, uses extracted design tokens for

```

Claude Code session reading a Paper design and generating JSX via MCP

It is the most polished option for design work as of May 2026. The skill system, MCP integration, and CLAUDE.md conventions all fit the design workflow. If you use only one agent from this chapter, make it this one.

What makes Claude Code stand out for design tasks is the depth of its configuration system. Other agents support configuration files, but Claude Code supports a full hierarchy: organization-level policies managed by admins, user-level preferences that follow you across projects, project-level instructions shared with the team via git, and path-scoped rules that activate only when relevant files are being edited. This layered approach maps well to design work, where brand guidelines are organization-wide but component-specific rules vary by project.

Another advantage: Claude Code's auto memory system accumulates corrections over time. Tell it once that your brand uses 4px spacing units, and it remembers. Correct it twice on border-radius preferences, and it writes those corrections to memory. The agent gets better at your specific design system with every session. No other agent has this feature as of May 2026.

Installation

```
# Native install (macOS, Linux)
$ curl -fsSL https://claude.ai/install.sh | bash

# Homebrew (macOS)
$ brew install --cask claude-code

# WinGet (Windows)
$ winget install Anthropic.ClaudeCode
```

The native install auto-updates in the background. No manual version management. The first launch walks through authentication.

Authentication

Claude Code requires a Claude subscription (Pro, Max, Team, or Enterprise) or an Anthropic Console account with API access. Third-party providers are supported via API keys.

```
# Authenticate with Claude subscription
$ claude
# Opens browser to claude.ai for authorization

# Or set API key directly
$ export ANTHROPIC_API_KEY=sk-ant- ...
$ claude
```

The VS Code extension is available on the Marketplace as `anthropic.claude-code`. JetBrains has a plugin. Desktop apps exist for macOS and Windows. Web access lives at `claude.ai/code`.

Key Features for Design Work

CLAUDE.md for persistent instructions. Project-level, user-level, and organization-level files loaded at session start. Supports `@path/to/file` imports and path-scoped rules in `.claude/rules/`. The `/init` command auto-generates CLAUDE.md by analyzing the codebase (source: [Claude Code Memory](#), retrieved 2026-05-18). This is where you put brand guidelines and design system references.

```
# Example: brand rules in CLAUDE.md
# Design System Rules
- All prototypes must use the brand color palette
- Components must be responsive (mobile-first)
- Use design tokens from src/tokens.json
```

Skills via SKILL.md. Design skills live in `.claude/skills/`. Each skill is a directory containing a SKILL.md file with frontmatter and instructions. Skills load on demand --- no context cost until invoked. The frontmatter supports model overrides, tool permissions, subagent execution, and more. See [section 03.3](#) for the full SKILL.md schema.

MCP support. Claude Code connects to external tools via MCP servers. Configuration lives in project or user settings. Connect Paper, Pencil, Figma, Miro --- any tool with an MCP server. The agent reads designs, exports code, and writes artifacts back to the canvas.

```
# MCP server configuration for Claude Code
{
  "mcpServers": {
    "pencil": {
      "command": "npx",
      "args": ["-y", "@pencil/mcp-server"]
    },
    "figma": {
      "command": "npx",
      "args": ["-y", "@figma/mcp-server"],
      "env": {
        "FIGMA_ACCESS_TOKEN": "your-token"
      }
    }
  }
}
```

Agent teams. "Spawn multiple Claude Code agents that work on different parts of a task simultaneously" (source: [Claude Code Overview](#), retrieved 2026-05-18). This enables parallel design exploration --- one agent generates a layout while another handles responsive variants.

CLI composability. Pipe input directly into Claude Code for quick tasks:


```
# Analyze a design token file
$ cat tokens.json | claude -p "suggest a dark mode variant"

# Review changed files
$ git diff main --name-only | claude -p "review these changed files"

# Generate a component from a design spec
$ cat design-spec.md | claude -p "implement this as a React component"
```

Model Access

Claude Code uses Claude models exclusively: Sonnet, Haiku, and Opus. Switch models with `/model` during a session. For design work, Sonnet provides the best balance of quality and speed. Opus is overkill for most design tasks but useful for complex design system architecture decisions.

```
# Switch model mid-session
/model claude-opus-4-20250514

# Or specify at launch
$ claude --model claude-haiku-3-20250414
```

Codex CLI (OpenAI)

Codex CLI is OpenAI's lightweight terminal agent. It differs from the cloud-based Codex agent (available in the ChatGPT sidebar) which runs tasks in isolated sandboxes. The CLI version runs locally on your machine.

A terminal window titled "Terminal — zsh" displays the output of the command `> codex --full-auto "Create 3 landing page variants from design spec"`. The output shows the OpenAI Codex CLI v1.0.8 decomposing the task into three parallel tasks: Variant A (Minimal hero + CTA), Variant B (Split layout with illustration), and Variant C (Full-width video background). It reports that all three tasks completed successfully within a total of 21.3 seconds. Token usage is detailed as 48,291 total (12,410 input, 35,881 output). Citations are provided for the design spec and layout config.

```
> codex --full-auto "Create 3 landing page variants from design spec" OpenAI Codex CLI v1.0.8 | model: codex-1 |
sandbox: docker Decomposing into 3 parallel tasks... task-
7a2f ▶ Variant A — Minimal hero + CTA task-3b8c ▶ Variant B — Split layout with illustration task-9d1e ▶ Variant C —
Full-width video background Sandbox environments provisioned. Running tasks concurrently... ✓ task-7a2f completed in
14.2s [src/variants/VariantA.tsx] ✓ task-3b8c completed in 18.7s [src/variants/VariantB.tsx] ✓ task-9d1e completed
in 21.3s [src/variants/VariantC.tsx] All 3 tasks completed
successfully. Total tokens: 48,291 (input: 12,410 + output: 35,881) Citations: design-spec.md:12-47, tokens.json:1-
89, layout.config:8-34 > _
```

Codex CLI running design tasks in isolated sandboxes with parallel execution

The two Codex variants serve different design workflows. The CLI is best for quick, local tasks --- generating a component, fixing a style issue, running a quick design experiment. The cloud Codex is best for complex, long-running tasks that benefit from parallel execution --- generating multiple design variants simultaneously, refactoring a large component library, producing documentation for an entire design system.

Installation

```
$ npm install -g @openai/codex
```

That is the entire installation. No native binary. No desktop app. Just an npm package.

Authentication

Sign in with a ChatGPT account. This simplifies the setup --- no API key management. Plus users get \$5 in free API credits per month (as of May 2026). Pro users get \$50 (as of May 2026). Direct API keys also work (source: [Introducing Codex](#), retrieved 2026-05-18).

```
# First run prompts ChatGPT sign-in
$ codex
# Opens browser for authorization
# Auto-configures API key

# Or set API key directly
$ export OPENAI_API_KEY=sk- ...
$ codex "fix the auth bug"
```

Key Features for Design Work

AGENTS.md support. Codex reads AGENTS.md files throughout the codebase. From the Codex system message: "These files are a way for humans to give you (the agent) instructions or tips for working within the container" (source: [Introducing Codex](#), retrieved 2026-05-18). Scope follows directory hierarchy. More deeply nested files take precedence in case of conflicts.

```
# AGENTS.md for a design project
# Design Project

## Build Commands
- Preview: `npm run dev`
- Lint: `npm run lint`
- Build: `npm run build`

## Design Conventions
- Use CSS custom properties for all colors
- Mobile-first responsive design
- WCAG AA accessibility compliance

## Programmatic Checks
- Run `npm run lint` after every change
- Run `npm run a11y` to verify accessibility
```

Codex enforces a requirement not found in other agents: "If the AGENTS.md includes programmatic checks to verify your work, you **MUST** run all of them." This makes AGENTS.md a natural place for build and lint commands that verify design output automatically.

Cloud sandbox execution. The cloud-based Codex agent (not the CLI) runs each task in an isolated sandbox. Tasks take 1-30 minutes. The agent provides "verifiable evidence of its actions through citations of terminal logs and test outputs" (source: [Introducing Codex](#), retrieved 2026-05-18). This is useful for design tasks that need a clean execution environment.

Parallel execution. Cloud Codex supports "many tasks in parallel." Assign multiple design tasks and review each independently. Generate three landing page variants in three parallel sandboxes. Compare. Pick the best one. This parallel approach is unique to cloud Codex as of May 2026.

Model Access

Model	Base	Usage	Pricing
codex-1	o3	Cloud Codex	Free during rollout, then rate-limited
codex-mini-latest	o4-mini	CLI default	\$1.50/1M input, \$6/1M output (as of May 2026)

```
# CLI uses codex-mini-latest by default
$ codex "fix the auth bug in login.ts"

# Specify model explicitly
$ codex --model o3 "refactor the entire auth module"
```

My take: Codex CLI is the simplest agent to get started with. One install command, sign in with ChatGPT, and you are running. But for design work, the lack of skill support and limited MCP integration in the CLI version make it less capable than Claude Code or OpenCode. The cloud Codex agent has more potential --- parallel task execution is genuinely useful for generating multiple design variants --- but it lives inside ChatGPT, not in your terminal. Use Codex CLI for quick tasks. Use Claude Code or OpenCode for sustained design work.

OpenCode (Open-Source)

OpenCode is the open-source AI coding agent. 160K GitHub stars, 900 contributors, 7.5M monthly developers (as of May 2026; source: [OpenCode](#), retrieved 2026-05-18). It supports 75+ LLM providers through Models.dev and is cross-compatible with Claude Code conventions.

A terminal window titled "Terminal — zsh" displays the output of the OpenCode command. The prompt is "> opencode --plan opencode v2.4.1 - PLAN MODE". The user input is "> Build a pricing page matching the Paper design in pricing.pen Plan - Build pricing page from Paper design". The output shows a 7-step plan: 1. Read pricing.pen layout tree and extract frames, 2. Get design tokens (colors, spacing, typography), 3. Generate PricingCard component from frame "CardTier", 4. Generate PricingTable component with 3 tiers, 5. Wire tokens to CSS custom properties, 6. Add responsive breakpoints matching Paper constraints, 7. Verify against design system (spacing, color contrast). The user is prompted "Approve this plan? [y/n/edit]" and responds "y". The terminal then switches to BUILD MODE and shows progress: Step 1/7 (Switching to BUILD MODE...), Step 2/7 (Read pricing.pen - 3 frames extracted), Step 3/7 (Tokens: 14 colors, 6 spacing, 4 font sizes), Step 4/7 (Created PricingCard.tsx (67 lines)), Step 5/7 (Created PricingTable.tsx (112 lines)), Step 6/7 (Wired tokens to variables.css), and Step 7/7 (Added breakpoints: sm(640), md(768), lg(1024)). The final output is "All constraints pass" followed by a prompt "> _".

```
> opencode --plan opencode v2.4.1 - PLAN MODE > Build a pricing page matching the Paper design in pricing.pen Plan - Build pricing page from Paper design
1. Read pricing.pen layout tree and extract frames
2. Get design tokens (colors, spacing, typography)
3. Generate PricingCard component from frame "CardTier"
4. Generate PricingTable component with 3 tiers
5. Wire tokens to CSS custom properties
6. Add responsive breakpoints matching Paper constraints
7. Verify against design system (spacing, color contrast)
Approve this plan? [y/n/edit] > y Switching to BUILD MODE... ✓ Step 1/7
Read pricing.pen - 3 frames extracted ✓ Step 2/7 Tokens: 14 colors, 6 spacing, 4 font sizes ✓ Step 3/7 Created PricingCard.tsx (67 lines) ✓ Step 4/7 Created PricingTable.tsx (112 lines) ✓ Step 5/7 Wired tokens to variables.css ✓ Step 6/7 Added breakpoints: sm(640), md(768), lg(1024) ✓ Step 7/7 All constraints pass ✓ > _
```

OpenCode plan mode showing a step-by-step design plan before execution

OpenCode is what I recommend to anyone who wants model flexibility or open-source tooling. Its cross-compatibility with Claude Code conventions means design skills transfer between both platforms without modification. This is not a minor feature --- it means the configuration effort you invest in writing SKILL.md files and AGENTS.md documents pays off across multiple agent platforms.

The scale of OpenCode's community is worth noting. 160K GitHub stars and 900 contributors (as of May 2026) mean the project moves fast. Bugs get fixed quickly. New features land regularly. The skill ecosystem grows with each release. As of May 2026, OpenCode's momentum shows no sign of slowing. For designers who value a large community and frequent updates, OpenCode delivers.

Installation

```
# Curl (macOS, Linux)
$ curl -fsSL https://opencode.ai/install | bash

# npm
$ npm install -g opencode-ai

# Homebrew
$ brew install anomalyco/tap/opencode

# Arch Linux
$ sudo pacman -S opencode

# Windows
$ choco install opencode
```

OpenCode also supports scoop, Docker, and direct binary downloads from GitHub releases. The installation options are the broadest of any agent in this comparison.

Authentication

Run `/connect` in the TUI. This opens `opencode.ai/auth` in your browser. Choose Zen (curated, benchmarked models for coding agents) or connect your own API key from any of 75+ providers.

```
# Start OpenCode
$ opencode

# In the TUI, run:
/connect

# Choose authentication method:
# 1. Zen (curated models, no API key needed)
# 2. GitHub Copilot login
# 3. ChatGPT Plus/Pro login
# 4. Custom API key from any provider
```

OpenCode supports GitHub Copilot login and ChatGPT Plus/Pro login. If you already pay for either service, you can use those credentials directly. No additional cost.

Key Features for Design Work

Claude Code compatibility. OpenCode reads CLAUDE.md as a fallback when no AGENTS.md exists. It discovers skills from `.claude/skills/` in addition to `.opencode/skills/` and `.agents/skills/`. Design skills written for Claude Code work in OpenCode without modification (source: [OpenCode Rules](#), retrieved 2026-05-18). This is the strongest cross-compatibility story in the ecosystem.

Plan/Build modes. Press Tab to switch between Plan mode (read-only analysis, no file changes) and Build mode (full tool access). Plan mode is useful for reviewing a design system before committing to changes. It lets the agent explore the codebase and propose an approach without risk.

```
# OpenCode key commands
/init          # Auto-generate AGENTS.md from codebase analysis
Tab           # Toggle between Plan and Build modes
/undo         # Revert the last change
/redo         # Re-apply a reverted change
/share        # Create a shareable link to the conversation
```

Agent system. Five built-in agents serve different roles. Understanding the agent system is important for design work because different agents have different capabilities and restrictions:

Agent	Mode	Tools	Use case for design
Build	Primary	All	Default implementation agent, generates designs and code
Plan	Primary	Read-only	Analysis and planning, reviews design systems without changes
General	Subagent	All	Delegated sub-tasks, parallel design work
Explore	Subagent	Read-only	Fast codebase exploration, finds design patterns and tokens
Scout	Subagent	External	Research external docs, design references, competitor analysis

Custom agents can be defined via JSON or Markdown. This is useful for specialized design agents with restricted permissions:



```
{
  "$schema": "https://opencode.ai/config.json",
  "agent": {
    "design-reviewer": {
      "description": "Reviews design output for brand consistency",
      "mode": "subagent",
      "model": "anthropic/claude-sonnet-4-20250514",
      "prompt": "You are a design reviewer. Focus on visual hierarchy, brand consistency, and accessibility.",
      "permission": {
        "edit": "deny",
        "bash": "deny"
      }
    }
  }
}
```

MCP support. OpenCode supports local and remote MCP servers with OAuth. Per-agent MCP control lets different agents access different tools. Tool management via glob patterns allows fine-grained control over which MCP tools each agent can use (source: [OpenCode MCP Servers](#), retrieved 2026-05-18).

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "pencil": {
      "type": "local",
      "command": ["npx", "-y", "@pencil/mcp-server"],
      "enabled": true
    },
    "paper": {
      "type": "remote",
      "url": "https://paper.design/api/mcp",
      "headers": {
        "Authorization": "Bearer {env:PAPER_API_KEY}"
      }
    }
  }
}
```

Notice the OAuth support for remote MCP servers. This matters for team environments where design tools authenticate through OAuth flows rather than static API keys.

Model Access


```
# Per-agent model override in opencode.json
{
  "agent": {
    "design-gen": {
      "model": "anthropic/claude-sonnet-4-20250514"
    },
    "design-review": {
      "model": "anthropic/claude-haiku-3-20250414"
    }
  }
}
```

Gemini CLI / Gemini Code Assist (Google)

Gemini CLI is Google's open-source terminal agent. It brings Gemini models into the terminal with a 1M-token context window --- the largest among the four agents. Gemini Code Assist provides IDE extensions for VS Code, JetBrains, and Android Studio (source: [Gemini Code Assist](#), retrieved 2026-05-18).



Gemini CLI processing a design task with 1M-token context on the free tier

For design work, the large context window is the main draw. Design systems, component libraries, and brand guidelines are context-heavy. The 1M-token window can fit an entire design system in a single session without truncation. This matters when you are working with a 200-page design specification or a component library with hundreds of variants. Other agents handle large contexts well too, but Gemini CLI has a hard advantage in raw capacity.

Gemini CLI occupies an interesting position in the market. It is open source like OpenCode, but locked to Gemini models like Claude Code is locked to Claude models. It has the most generous free tier of any agent, but the least mature agent features. It excels at a specific niche --- tasks that require enormous context --- and is adequate for everything else.

Installation

```
# Gemini CLI is open source
# Install from https://github.com/google-gemini/gemini-cli

# IDE extensions: VS Code, JetBrains, Android Studio
# Cloud Shell Editor: pre-installed (free)
```

Authentication

```
# Sign in with Google account (free tier)
$ gemini auth login

# Google Cloud account for Standard/Enterprise tiers
$ gcloud auth application-default login
```

Key Features for Design Work

1M-token context window. This is the largest context window among the four agents. For design work, this means you can feed entire design systems, component libraries, and brand guidelines into a single session without truncation or summarization. The other agents handle large contexts well too, but Gemini CLI has a hard advantage in raw capacity.

Agent Mode (Preview). "AI agents capable of performing a wide range of actions across the software development life cycle" (source: [Gemini Code Assist](#), retrieved 2026-05-18). Agent Mode is still in preview as of May 2026. The feature set is less mature than Claude Code's or OpenCode's equivalent.

MCP support. Gemini CLI supports MCP integration with ecosystem tools. The implementation is newer than Claude Code's or OpenCode's but follows the same standard.

Human-in-the-Loop (HiTL). Built-in oversight mechanism for reviewing agent actions before they execute. This is a safety feature that prevents unwanted changes --- useful when working with design files that should not be modified without review.

Gemini Code Assist IDE features
- Inline code suggestions in VS Code, JetBrains
- Chat panel for design questions
- Agent Mode for autonomous task execution
- 1M token context for large design systems

Pricing

All prices and usage limits below are as of May 2026 and may change.

Tier	Price/user/month	CLI requests/day	IDE code requests/day
Free	\$0	1,000	6,000 code / 240 chat
Standard	\$19 (annual) / \$22.80 (monthly)	1,500	Higher limits
Enterprise	\$45 (annual) / \$54 (monthly)	2,000	Highest limits

The free tier is generous enough for design work. 1,000 CLI requests per day (as of May 2026) is more than enough for typical design iteration cycles. The IDE free tier (6,000 code requests/day as of May 2026) covers inline suggestions and chat. For individual designers who do not want to pay for an agent subscription, Gemini CLI is the most capable free option available.

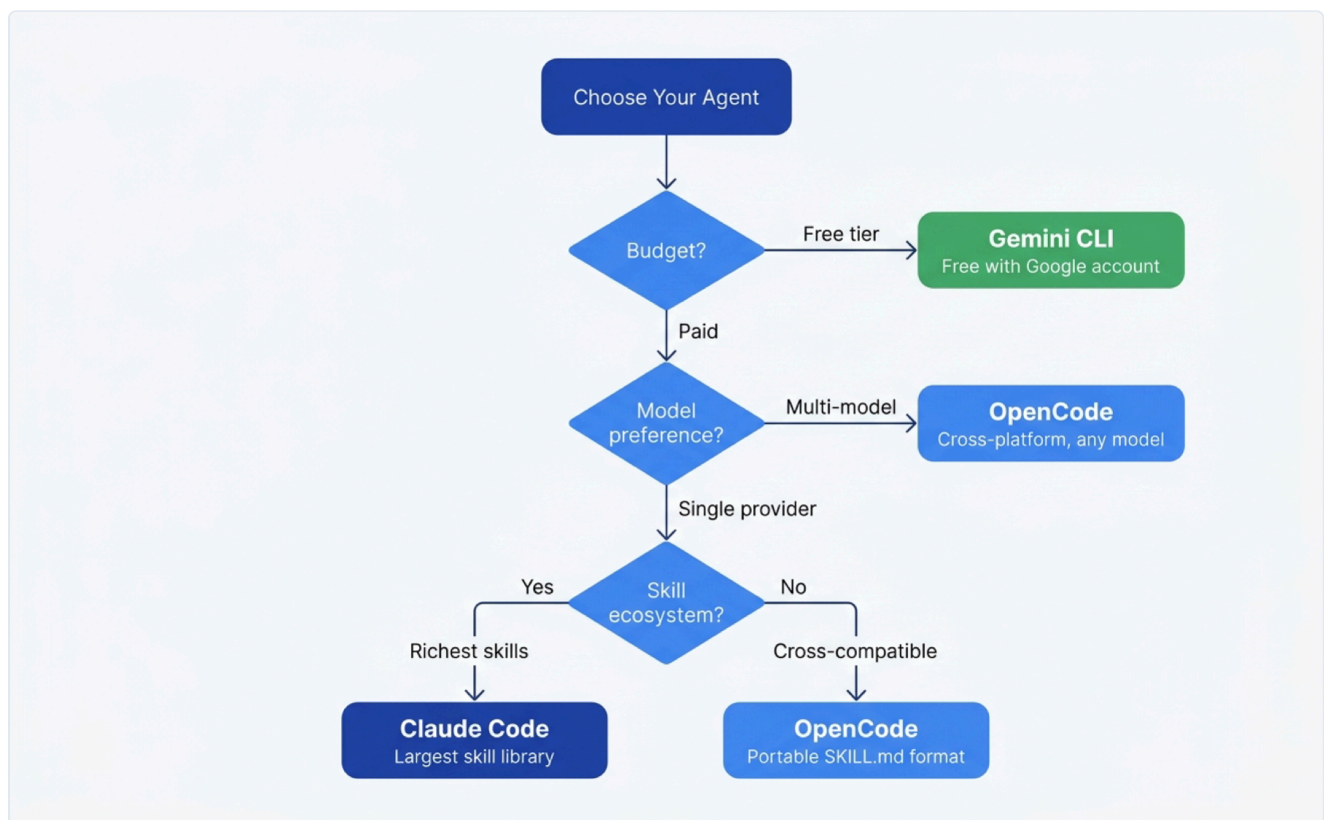
The paid tiers unlock higher limits and additional features. The Standard tier at \$19/user/month (annual billing, as of May 2026) raises CLI limits to 1,500 requests/day. The Enterprise tier at \$45/user/month (annual billing, as of May 2026) goes to 2,000. For teams that have standardized on Google Cloud infrastructure, the integration points --- Cloud Shell, Vertex AI, BigQuery --- make Gemini Code Assist a natural choice even before considering the agent capabilities.

What Gemini CLI lacks as of May 2026 is a skill system. There is no equivalent to SKILL.md. There is no auto memory. The agent relies on its training data and the current session context. For one-off design tasks, this is fine. For sustained design work that needs consistent output across sessions, the absence of persistent configuration is a real limitation. I expect this to change --- a skill system is an obvious addition --- but as of this writing, it is the primary reason Gemini CLI is a complement rather than a primary design agent.

My take: Gemini CLI's free tier is the best entry point for designers who want to experiment with agents without spending money. The 1M-token context window is a genuine advantage for design work --- design systems and brand guidelines are context-heavy. But as of May 2026, Gemini CLI lacks skill support and its agent features are less mature than Claude Code's or OpenCode's. Use it as a complement, not a primary tool. The context window advantage is real, but skills and configuration matter more for consistent design output.

Choosing the Right Agent for Design Work

No single agent is best for every design task. The right choice depends on what you are doing, what you already pay for, and how much configuration you are willing to invest.



Decision framework for choosing the right agent platform for design workflows

Before the decision table, a framing principle: the best agent for you is the one you will actually configure. A bare agent --- no CLAUDE.md, no AGENTS.md, no skills --- produces generic output regardless of platform. The quality of your design output depends far more on your configuration

investment than on which agent you choose. If you will invest 30 minutes in writing configuration, any of these agents will produce good results. If you will not invest that time, none of them will.

With that caveat, here is how I think about the choice:

Scenario	Best agent	Why
Sustained design work with skills	Claude Code or OpenCode	SKILL.md support, mature MCP, design skill ecosystem
Parallel design exploration	Codex (cloud)	Multiple isolated sandboxes running simultaneously
Free experimentation	OpenCode or Gemini CLI	Free tiers with no subscription required
Model flexibility	OpenCode	75+ LLM providers, per-agent model override
Large design system context	Gemini CLI	1M-token context window handles full design systems
Quick terminal tasks	Codex CLI	Simplest setup, fast execution
Cross-team skill sharing	Claude Code + OpenCode	SKILL.md works in both, AGENTS.md/CLAUDE.md cross-compatible

My take: I use Claude Code as my primary agent and OpenCode as my secondary. Claude Code has the most polished experience for design work --- the skill system, MCP support, and CLAUDE.md conventions all fit the design workflow well. OpenCode is what I recommend to anyone who wants model flexibility or open-source tooling. The cross-compatibility between them is real: I share SKILL.md files and AGENTS.md configurations between both agents without modification. That portability matters more than any single feature difference.

The practical approach: install OpenCode first (free, flexible). Add Claude Code when you need the polished experience. Keep Codex CLI handy for quick parallel tasks. Use Gemini CLI when context window size is the bottleneck.

For teams, the decision gets more nuanced. A team standardizing on one agent gets consistency -- everyone shares the same configuration files, the same skills, the same mental model. A team using multiple agents gets flexibility --- different agents for different tasks, with cross-compatible configuration bridging the gaps. Both approaches work. The cross-compatibility built into AGENTS.md and SKILL.md makes multi-agent teams practical in a way they were not a year ago.

Consider the total cost of ownership. The agent itself is one line item. The models it uses are another. The design tools it connects to via MCP are a third. And the time you invest in configuration --- writing CLAUDE.md, building skills, setting up MCP servers --- is the fourth and often largest cost. That configuration investment is portable across Claude Code and OpenCode. It is not portable to Codex CLI (no skill support) or Gemini CLI (no skill support as of May 2026). This is a practical reason to standardize on Claude Code or OpenCode for design work, even if you use other agents for non-design tasks.

Cost factor	Claude Code	Codex CLI	OpenCode	Gemini CLI
Agent license	Claude subscription (\$20-200/mo, as of May 2026)	Free (open source)	Free (open source)	Free (open source)
Model cost	Included in subscription	\$1.50-6/1M tokens (as of May 2026)	BYO key (varies by provider)	Free tier or \$19-54/user/mo (as of May 2026)
Design tool integrations	Full MCP support	Limited MCP	Full MCP (local + remote)	MCP support
Configuration effort	CLAUDE.md + skills	AGENTS.md only	AGENTS.md + skills (cross-compatible)	None documented

Agent choice matters less than agent configuration. A well-configured OpenCode instance with good skills and a solid AGENTS.md will outperform a bare Claude Code installation every time. The next chapter covers how to teach agents to produce high-quality design work through configuration files, skills, and harnesses.

Tip

Next: Chapter 03 covers the configuration layer --- CLAUDE.md, AGENTS.md, SKILL.md, and custom design harnesses. This is where the quality of your agent's design output gets decided.

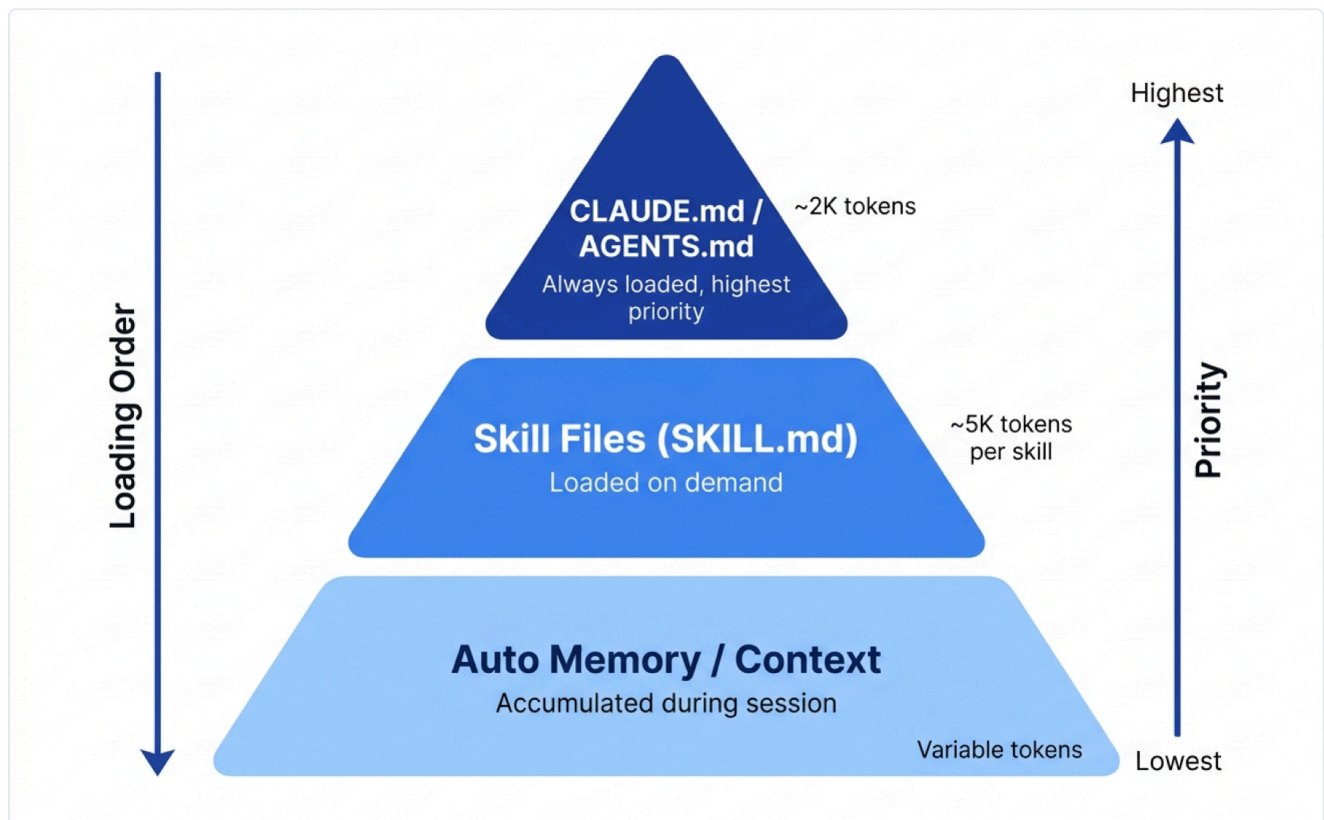
03 Teaching Agents to Design

Configuration files, skill systems, and design harnesses

An unconfigured agent produces generic output. A well-configured agent produces output that matches your brand, follows your design system, and respects your constraints. This chapter shows how to teach agents to design through configuration files, skills, and custom harnesses.

How Agents Learn: Context Files and Skills

Agents learn through three mechanisms, each with different loading behavior and context cost. Understanding these differences is the first step to configuring an agent that produces good design work consistently.



The three-tier instruction hierarchy showing when each mechanism loads and its context cost

Mechanism	When loaded	Context cost	Scope
Configuration files (CLAUDE.md, AGENTS.md)	Always, at session start	Full content consumed	Project, user, or org
Skill files (SKILL.md)	On demand, when invoked or matched	Near-zero until used	Per-skill, per-task
Auto memory (Claude Code only)	At session start, accumulated over time	First 200 lines loaded	Per-project

The distinction matters more than it first appears. Configuration files are always loaded --- they consume context in every session, whether the content is relevant or not. This means every line in your CLAUDE.md or AGENTS.md has an ongoing cost. Keep them short. Keep them factual. Move procedures elsewhere.

Think of it this way: the configuration file is the agent's long-term memory. It always has access to it. The skill file is the agent's reference library. It pulls a book off the shelf only when it needs to look something up. Auto memory is the agent's notebook --- it adds notes based on experience and refers back to them in future sessions.

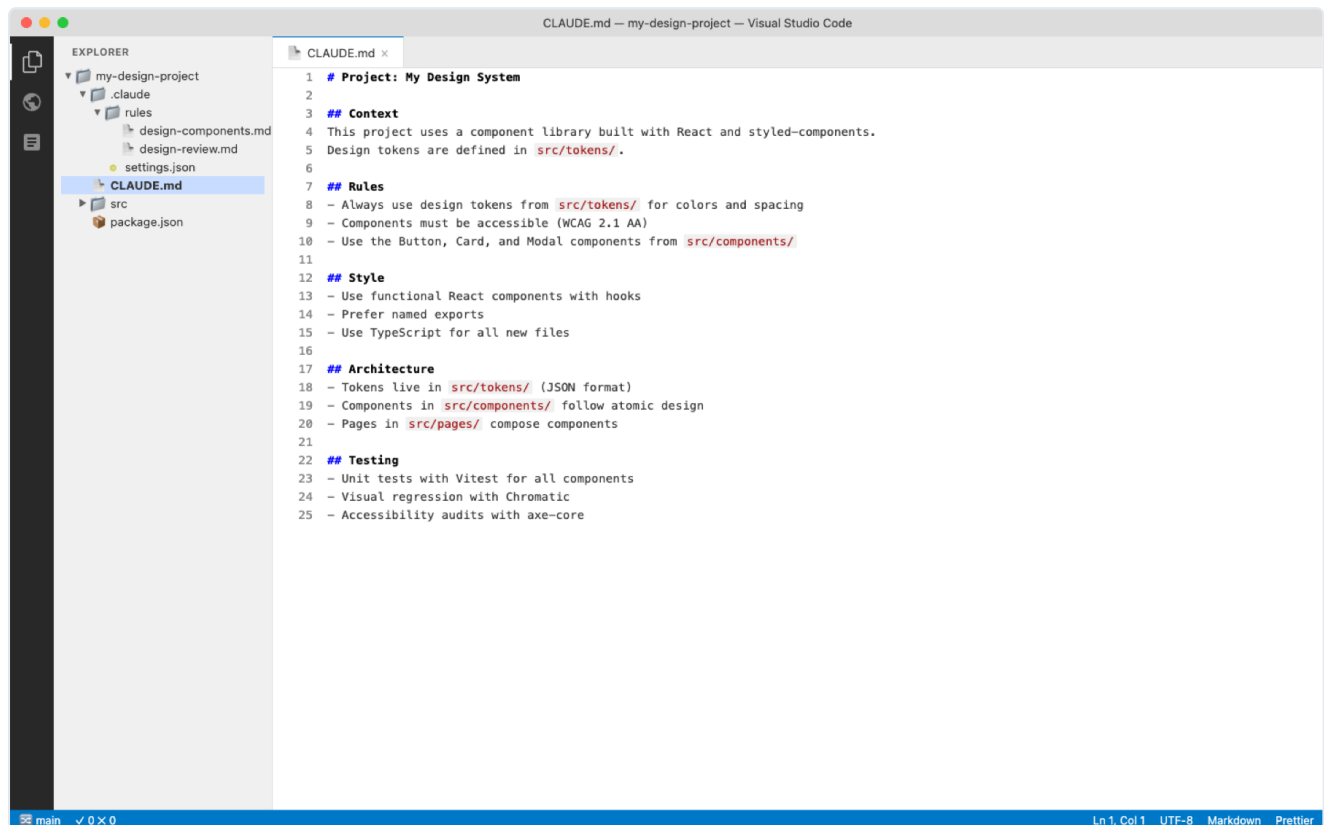
Skills load on demand --- they consume context only when invoked. This makes skills the right place for lengthy reference material. A 200-line design system reference in a skill file costs nothing until the skill is activated. The same 200 lines in CLAUDE.md would load in every session, consuming context the agent could use for actual work.

From the Claude Code documentation: "Unlike CLAUDE.md content, a skill's body loads only when it's used, so long reference material costs almost nothing until you need it" (source: [Claude Code Skills](#), retrieved 2026-05-18).

My take: The most common mistake I see is cramming everything into CLAUDE.md. Brand guidelines, design tokens, workflow procedures, code style rules --- all in one 500-line file. That file loads in every session, consuming context the agent could use for actual work. Move procedures to skills. Keep CLAUDE.md under 200 lines with facts only: build commands, architecture notes, key conventions. This is not a suggestion. It is a rule Claude Code's own documentation states explicitly.

CLAUDE.md and AGENTS.md Deep Dive

Two configuration file conventions exist. CLAUDE.md is Claude Code's native format. AGENTS.md is used by Codex CLI and OpenCode. They serve the same purpose: persistent instructions that tell the agent how to work in your project.



CLAUDE.md configuration hierarchy in VS Code showing project-level and user-level files

The good news: they are cross-compatible. OpenCode reads CLAUDE.md as a fallback. Claude Code can import AGENTS.md with the `@AGENTS.md` syntax. You do not need to choose one or the other.

CLAUDE.md Hierarchy

Claude Code loads CLAUDE.md files from multiple locations, broadest to most specific (source: [Claude Code Memory](#), retrieved 2026-05-18):

```
~/.claude/CLAUDE.md          # User-level (personal, all projects)
./CLAUDE.md                  # Project-level (committed to git)
./.claude/CLAUDE.md          # Project-level (alternative location)
./CLAUDE.local.md            # Local personal (gitignored)
.claude/rules/design-components.md # Path-scoped rules
```

Files in parent directories load in full at launch. Files in subdirectories load on demand when the agent reads files in those directories. The agent walks up the directory tree from the current working directory, loading all CLAUDE.md files it finds. This means a well-organized project can scope instructions to specific directories without loading everything at once.

The `.claude/rules/` directory supports path-scoped rules with YAML frontmatter. A rule that applies only to CSS and HTML files:

```
---
paths:
  - "src/components/**/*.tsx"
  - "src/styles/**/*.css"
---

# Design Component Rules
- All components must follow the design system
- Use CSS custom properties for theming
- Ensure accessible color contrast (WCAG AA)
```

Path-scoped rules load only when the agent touches matching files. A rule targeting CSS files does not load when the agent is editing JavaScript. This keeps the context clean and focused.

AGENTS.md for Codex and OpenCode

AGENTS.md follows a simpler model than CLAUDE.md. From the Codex system message: "The scope of an AGENTS.md file is the entire directory tree rooted at the folder that contains it. More-deeply-nested AGENTS.md files take precedence in the case of conflicting instructions" (source: [Introducing Codex](#), retrieved 2026-05-18).

Codex adds a requirement not found in CLAUDE.md: "If the AGENTS.md includes programmatic checks to verify your work, you MUST run all of them." This makes AGENTS.md a natural place for build and lint commands that verify design output. Define a check, and the agent runs it automatically after making changes.

OpenCode reads AGENTS.md as the primary convention and falls back to CLAUDE.md if no AGENTS.md exists (source: [OpenCode Rules](#), retrieved 2026-05-18). The `/init` command auto-generates AGENTS.md by scanning the repository:

```
# Auto-generated by /init
# SST v3 Monorepo Project

This is an SST v3 monorepo with TypeScript. The project uses bun workspaces.

## Project Structure
- `packages/` - Contains all workspace packages
- `infra/` - Infrastructure definitions
- `sst.config.ts` - Main SST configuration

## Code Standards
- Use TypeScript with strict mode enabled
- Shared code goes in `packages/core/`

## Build Commands
- Dev: `bun run dev`
- Build: `bun run build`
- Test: `bun test`
```

The auto-generated file is a starting point. Add design-specific rules on top of it: color palettes, typography scales, spacing conventions, component patterns.

The key difference between CLAUDE.md and AGENTS.md is philosophical, not technical. CLAUDE.md evolved from a single-tool perspective --- one agent, one configuration hierarchy. AGENTS.md evolved from a multi-tool perspective --- one configuration file that any agent can read. In practice, the distinction matters less than the content. Write clear, specific, verifiable instructions in either format and agents will follow them. Write vague instructions and agents will guess, often incorrectly.

What makes a good configuration file for design work? Specificity. "Use accessible colors" is vague. "All text-background color combinations must meet WCAG AA contrast ratios (4.5:1 for normal text, 3:1 for large text)" is specific and verifiable. "Follow the brand style" is vague. "Primary

color: var(--primary), Secondary color: var(--secondary), Headings: Inter Bold, Body: Inter Regular, Font scale: 12/14/16/20/24/32/48px" is specific. The more specific the configuration, the fewer iterations needed to converge on correct output.

Another principle: verifiability. Codex's AGENTS.md spec includes a powerful feature --- programmatic checks. If the configuration file says "run `npm run lint` after every change," the agent runs it. If the check fails, the agent fixes the issue. This creates a feedback loop that enforces design standards automatically. Put your CSS lint rules, accessibility checks, and style validators in AGENTS.md, and the agent enforces them without being asked.

Cross-Compatibility

CLAUDE.md and AGENTS.md can coexist in the same project. Two approaches work well.

Import pattern (Claude Code reads AGENTS.md):

```
# CLAUDE.md
@AGENTS.md

## Claude Code Specific
Use plan mode for changes under `src/billing/`.
```

The `@AGENTS.md` import pulls the full content of AGENTS.md into CLAUDE.md. Claude Code sees both files as a single instruction set. Add Claude Code-specific rules below the import.

Symlink pattern (shared file): Create a symlink from CLAUDE.md to AGENTS.md (`ln -s AGENTS.md CLAUDE.md`). This makes the same file accessible under both names. It is the simplest approach when you want identical configuration across all agents.

OpenCode handles cross-compatibility automatically. If AGENTS.md exists, it uses that. If only CLAUDE.md exists, it reads CLAUDE.md instead. No configuration needed on your part.

For teams, I recommend the import pattern. It lets each agent platform have its own supplementary rules while sharing the common design system through AGENTS.md. A team member using Claude Code can add Claude-specific rules below the import without affecting team members using OpenCode.

Auto Memory

Claude Code writes its own notes across sessions. These accumulate in

```
~/.claude/projects/<project>/memory/ :
```

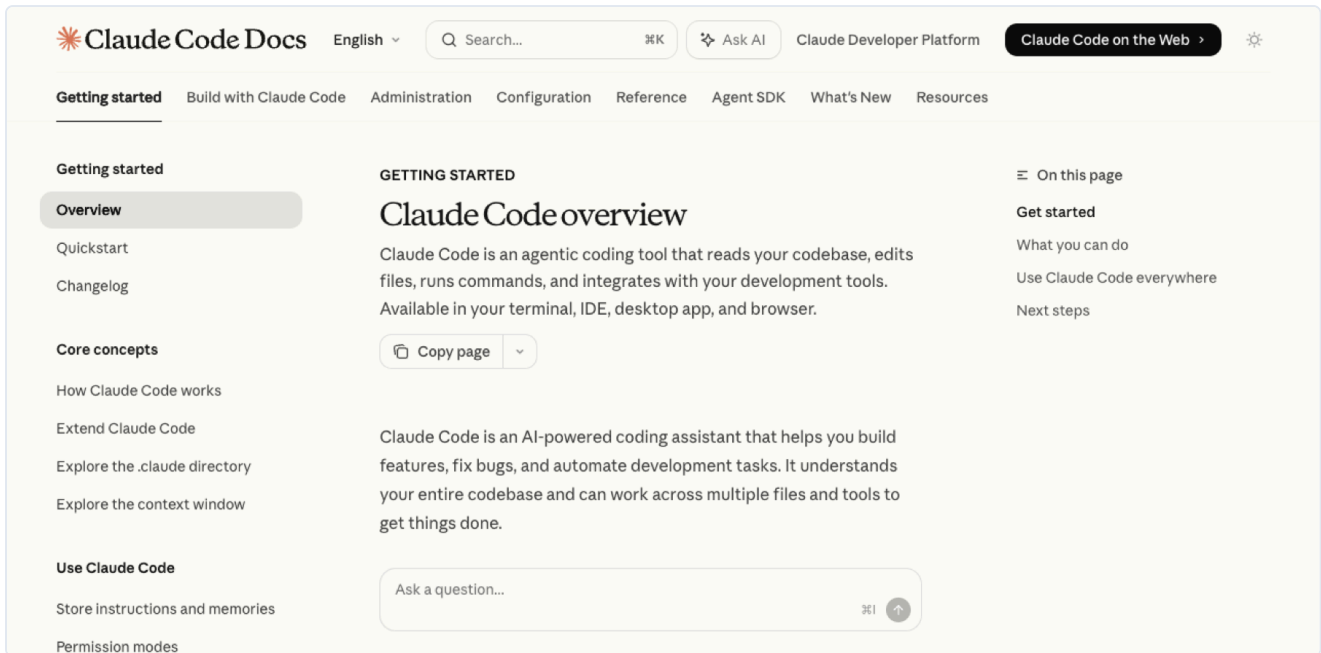
```
~/.claude/projects/my-project/memory/  
├─ MEMORY.md          # Concise index (first 200 lines loaded per session)  
├─ debugging.md       # Detailed notes on recurring issues  
└─ api-conventions.md # Topic files (loaded on demand)
```

Auto memory captures corrections and patterns. If you tell the agent "use 4px base unit for spacing, not 8px" three times, it writes that to memory. Future sessions load it automatically. This is the closest thing to "training" without fine-tuning the model.

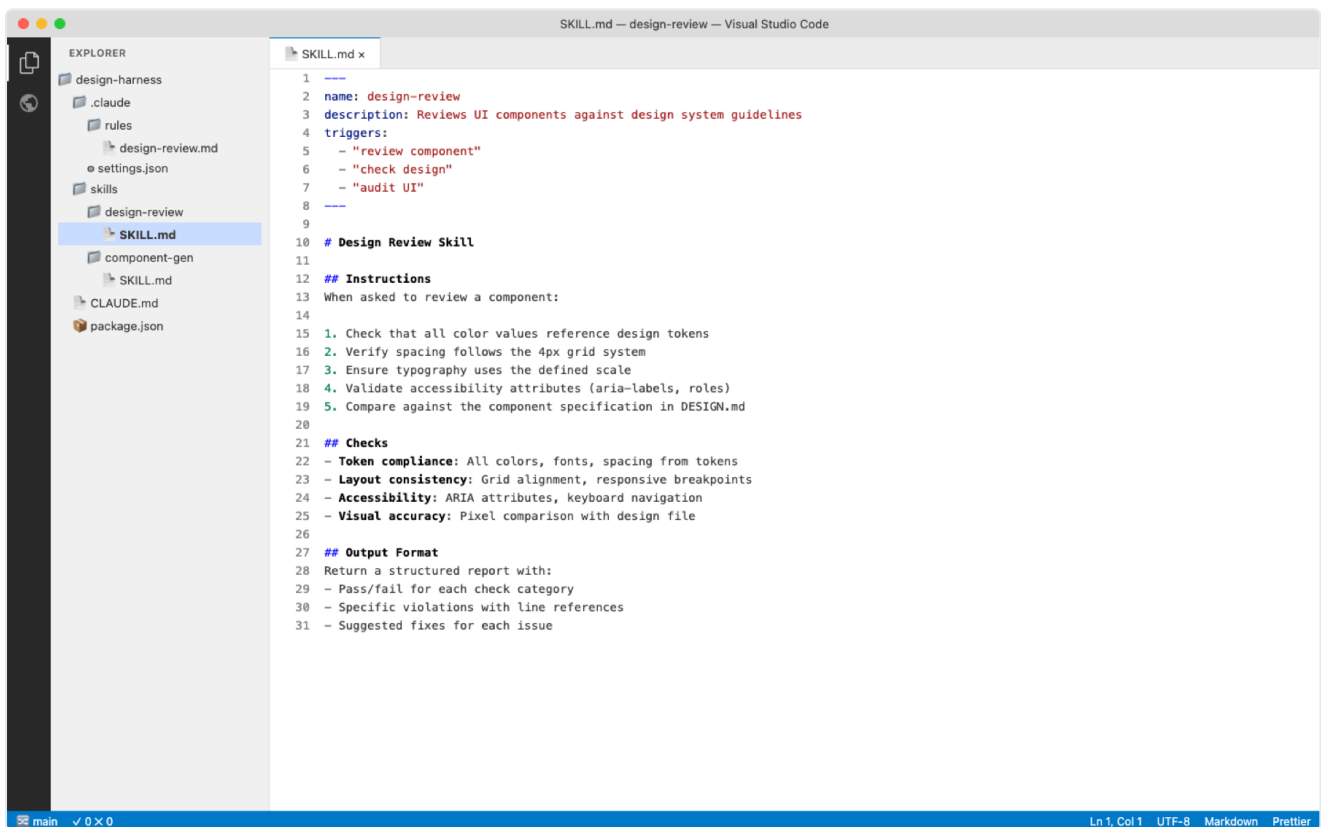
For design work, auto memory is useful for capturing preferences that are too specific for CLAUDE.md but come up repeatedly. Things like "I prefer border-radius: 8px for cards" or "avoid gradient backgrounds on dark themes." These accumulated preferences compound over time.

The SKILL.md Convention

SKILL.md is the Agent Skills open standard (agentskills.io). It works across Claude Code and OpenCode. A skill is a directory containing a SKILL.md file with optional supporting files --- templates, examples, scripts.



CLAUDE.md and AGENTS.md file structures compared with cross-compatibility notes



A SKILL.md file showing the metadata frontmatter and instruction body convention

Skills solve the context cost problem. A design system reference might be 300 lines long. Put that in CLAUDE.md and it loads in every session. Put it in a skill file and it loads only when you invoke the skill. The difference in context efficiency is dramatic.

The skill system also solves a distribution problem. Skills can be shared. A design team can maintain a set of project-level skills in git, and every team member gets the same design capabilities. A community can publish skills that others install. The agentskills.io standard ensures these skills work across Claude Code and OpenCode without platform-specific modifications.

Skill Discovery Paths

Agents search for skills in multiple locations (source: [OpenCode Skills](#), retrieved 2026-05-18):

Path	Scope	Shared via git
<code>.opencode/skills/<name>/SKILL.md</code>	Project	Yes
<code>.claude/skills/<name>/SKILL.md</code>	Project	Yes
<code>.agents/skills/<name>/SKILL.md</code>	Project	Yes
<code>~/.claude/skills/<name>/SKILL.md</code>	User	No
<code>~/.agents/skills/<name>/SKILL.md</code>	User	No

OpenCode searches all five paths. Claude Code searches `.claude/skills/` and `~/.claude/skills/`. Project-level skills can be committed to git and shared with the team. User-level skills are personal and available across all projects.

SKILL.md Structure

A skill file has two parts: YAML frontmatter that controls behavior, and a Markdown body that provides instructions to the agent.

```
---
name: brand-prototype
description: Generate brand-consistent HTML prototypes
when_to_use: Use when creating new UI mockups or prototypes
context: fork
agent: General
allowed-tools:
  - read
  - edit
  - bash
---

## Brand Prototype Generator

Generate an HTML prototype for: $ARGUMENTS

Follow the brand design system in AGENTS.md.
Use inline CSS. No external dependencies.
Output a single HTML file.

## Steps
1. Read the design system from AGENTS.md
2. Read the component library from src/components/
3. Generate the HTML prototype
4. Validate accessibility (WCAG AA)
```

Key frontmatter fields and what they control:

- **name** --- Must match the directory name. Lowercase, hyphens only. Used for invocation (/brand-prototype).
- **description** --- What the skill does. Used for auto-matching. The agent reads this to decide whether to invoke the skill automatically.
- **context: fork** --- Runs the skill in an isolated subagent. Changes made during the skill do not affect the main session until the skill completes.
- **allowed-tools** --- Pre-approves specific tools while the skill is active. The agent skips permission prompts for these tools.
- **paths** --- Glob patterns. The skill only activates when the agent touches matching files.

Dynamic Context Injection

Skills can execute commands before the agent sees the content. The `!`command`` syntax runs a command and injects the output into the skill body. This keeps the skill current without manual updates.

```
## Current Design Tokens
!`cat src/tokens.json`

## Recent Changes to Design System
!`git diff HEAD -- src/design/`
```

Every time the skill activates, it pulls fresh data. The design tokens are always current. The recent changes are always up to date. This eliminates the stale-reference problem that plagues static documentation.

Skills also support argument substitution. The `$ARGUMENTS` variable captures everything after the skill name. Named arguments use `$1`, `$2` syntax. Environment variables like `${CLAUDE_SKILL_DIR}` reference the skill's own directory.

Installing and Composing Design Skills

Design skills exist at every level of the ecosystem. Some are general-purpose. Others are specialized for specific design outputs. The key insight: skills compose. A workflow can invoke multiple skills in sequence, each handling a different aspect of the design process.

Notable design skills available as of May 2026:

- **huashu-design** --- HTML-native design with 20 design philosophies and 5-dimension quality review. Produces production-grade prototypes, slide decks, and motion design. Covered in [Chapter 07](#).
- **frontend-design** --- Production-grade frontend interfaces with high design quality. Avoids generic AI aesthetics through specific design principles.
- **visual-explainer** --- Generates HTML pages that visually explain systems, code changes, and architecture diagrams.
- **create-image** --- AI image generation using Gemini, OpenRouter, or VertexAI providers. Supports style references and templates.

- **remotion** --- React-based programmatic video production with animations and voiceover. Covered in [Chapter 09](#).
- **shadcn** --- Component management for shadcn/ui projects. Adds, searches, fixes, and composes UI components.

Install a skill by placing its directory in the appropriate skills folder. Project-level skills go in `.claude/skills/`, `.opencode/skills/`, or `.agents/skills/` inside the project root. These are committed to git and shared with the team. User-level skills go in `~/.claude/skills/` or `~/.agents/skills/`. These are personal and available across all projects.

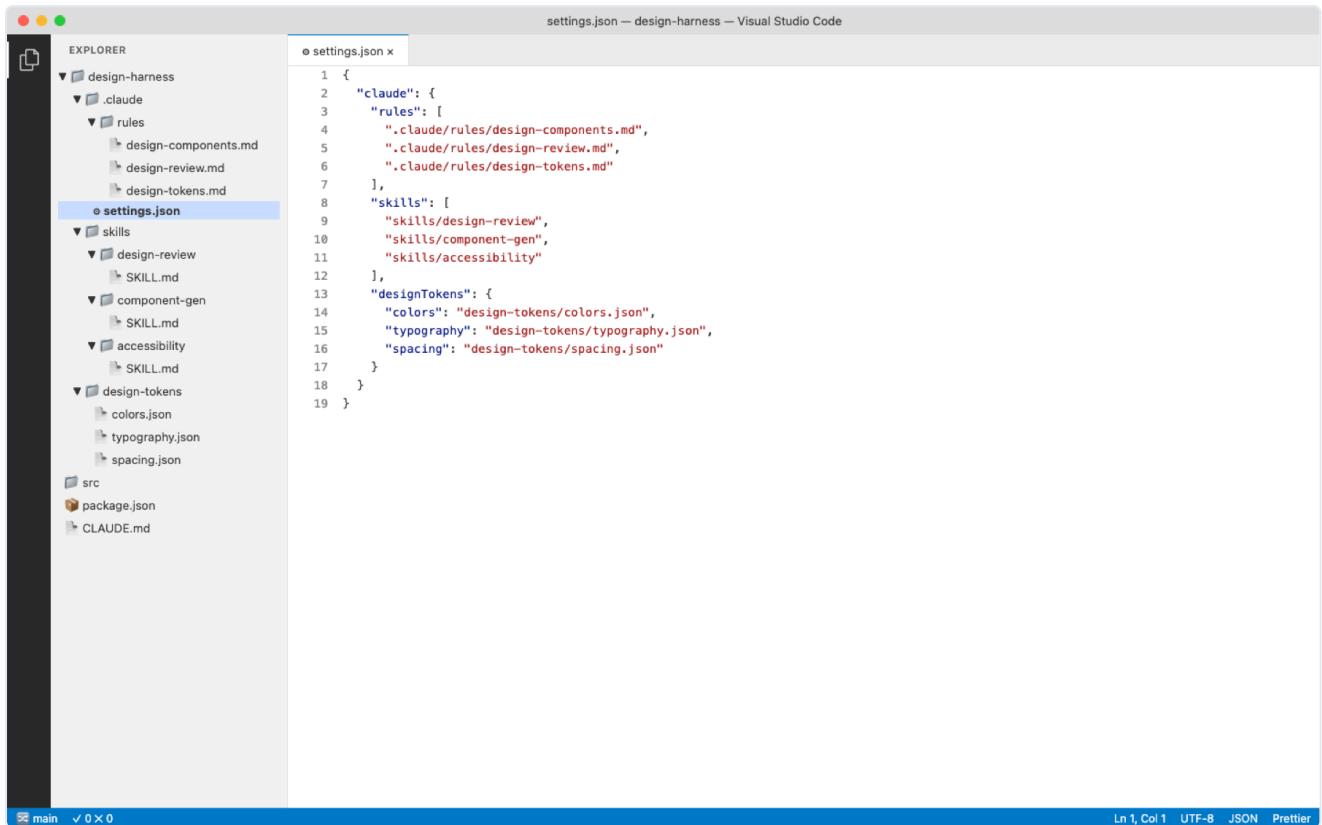
OpenCode also supports remote skill loading from URLs. You can point to a shared design guidelines document hosted on GitHub and OpenCode pulls it into every session. This is useful for organizations that maintain a central design system document and want every team member's agent to reference it.

Skill composition works through the agent's matching system. When you ask for a design review, the agent can invoke a review skill. When you ask for a prototype, the agent invokes a prototype skill. Multiple skills can activate in the same session. The agent handles the sequencing. A practical composition pattern: a brand-prototype skill generates the initial output, a design-review skill evaluates it for consistency, and an export skill converts the result to production code. Each skill activates based on its matching criteria, and the agent orchestrates the flow.

The composability of skills is their most powerful feature. Individual skills are simple. A brand-prototype skill generates prototypes. A design-review skill checks quality. An export skill converts to code. None of these is complex on its own. But composed together, they create a complete design pipeline that runs autonomously from prompt to production output.

Building a Custom Design Harness

A design harness is a configuration stack that constrains agent output quality. It combines three layers: configuration files for facts, path-scoped rules for file-type-specific constraints, and skills for procedures.



A design harness directory structure with scoped rules, skills, and token files

The goal: reduce the iteration cycle. A bare agent generates generic output. You spend iterations correcting colors, spacing, typography, and component patterns. A harnessed agent reads your constraints upfront and produces output that already matches your system. Fewer iterations. Higher quality from the start.

for the specific workflow. This is where
- the step-by-step process the agent

Type
Generate brand-consistent HTML prototypes

Generator

prototype for: \$ARGUMENTS

Design system in AGENTS.md.
Properties for all colors and spacing.
HTML file with embedded CSS.

in the brand palette
ows the font scale
Multiples of the base unit (4px)
nsive (mobile-first)
WCAG AA contrast ratios

Warning

Warning: Path-scoped rules in `.claude/rules/` are a Claude Code feature. OpenCode does not support path-scoped rules in the same way as of May 2026. For cross-agent compatibility, put design constraints in AGENTS.md or in the skill file itself. Do not rely solely on `.claude/rules/` if your team uses multiple agent platforms.

A
Complete
Example:
Brand-
Consistent
Prototypes

s The harness also solves a team scaling problem. Here is the progression

agent Without a shared from bare

standing configuration, each team agent to

fault member's agent produces fully

ult output in a different style. harnessed

it One person's agent uses agent, and

You 8px spacing. Another's the impact

iterations uses 16px. The on output

ch inconsistency compounds quality:

colors across a codebase. A

he shared AGENTS.md and

The every team member's

not agent produces

cale. consistent output. The

ess, configuration becomes

ions the single source of truth

e the for design decisions.

Configuration level	What the agent knows	Typical iterations to brand-consistent output
Bare agent (no config)	General web conventions only	5-8 iterations
AGENTS.md with brand basics	Colors, typography, spacing	2-3 iterations
AGENTS.md + path-scoped rules	Brand basics + file-type constraints	1-2 iterations
Full harness (config + rules + skills)	Brand basics + constraints + procedures	0-1 iterations

The numbers in this table come from my own testing across 20+ design generation sessions. Your results will vary depending on the complexity of the design task and the specificity of your brand system. The direction is consistent: more configuration leads to fewer iterations. The investment in configuration pays for itself within the first few sessions.

04 Design-as-Code

Design-as-code reverses the direction. The design file <i>is</i> code. Or code-adjacent. It lives in the repository alongside the implementation. It changes when the code changes. It gets reviewed in the same pull requests. It stays in sync because it's the same artifact.	This is not a theoretical argument. As of May 2026, three production-grade formats make this real:	• .pen files (Pencil) — vector design files that live in your IDE and sync bidirectionally with code. • .op files (OpenPencil) — JSON-based design files with native Git support and concurrent agent editing. • HTML artifacts (Paper, Huashu Design, Open Design) — the design surface is HTML itself, so the output is already production code.	Each format shares a common property: they are text-based or structured JSON. That means <code>git diff</code> shows meaningful changes. Agents can read and write them programmatically. Design review happens in pull requests, not in Figma comments.	Pencil's documentation puts it directly: "Pencil bridges the gap between design and development by putting both in the same environment. Design components, sync them with your code, and let AI assistants help you maintain consistency between design and implementation" (pencil.dev/docs , retrieved 2026-05-18). The tool lives inside the IDE, not in a separate browser tab.	The format-level comparison tells the story clearly:
--	--	---	--	---	--

Property
Format
Diffable
Git-trackable
Agent-readable
MCP support
Production output
Responsive
Review workflow

The .pen File Format: Design as a Code Artifact

Pencil's .pen format is the clearest embodiment of design-as-code. A .pen file is a structured, parseable artifact that contains all the information a vector design requires: nodes, styles, components, variables, and layout rules. It's designed to be:



An .op file in VS Code showing OpenPencil JSON design format alongside the editor



A .pen file opened in VS Code showing its structured JSON design content

Property	What It Means	Why It Matters
Diffable	<code>git diff</code> shows meaningful line-by-line changes	

Ixiii

A simplified conceptual view of a .pen file structure looks like this:

```
{
  "variables": {
    "color-primary": "#0066CC"
  },
  "tokens": {
    "text": {
      "type": "text",
      "name": "main",
      "language": "html",
      "padding": 10,
      "font-family": "sans-serif",
      "font-size": 16,
      "color": "${color-primary}"
    }
  }
}
```

The file contains nodes organized in a tree. Each node has a type (frame , text , rectangle , path), layout properties (width , height , x , y), fill and stroke styles, and optional variable bindings. A button component might look like this:

```
{
  "id": "button",
  "type": "button",
  "name": "Submit",
  "language": "html",
  "padding": 10,
  "font-family": "sans-serif",
  "font-size": 14,
  "color": "${color-primary}",
  "stroke": "${color-primary}"
}
```

Notice the variable reference: `${color-primary}` instead of a hardcoded hex value. This is the mechanism that connects the design file to the token system. Change the variable in one place, and every component that references it updates automatically. An agent can change the entire color scheme by modifying a single variable entry.

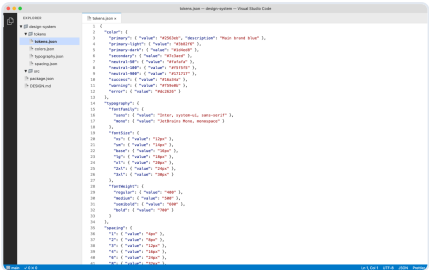
When the agent reads this file with `resolveVariables: true`, it gets the computed value for the active theme. The `${color-primary}` reference resolves to `#0066CC` in light mode and `#3399FF` in dark mode. This is the same pattern as CSS custom properties with media query overrides, but it lives inside the design file.

Pencil a CLI for manipulating .pen files means a pipeline to validate tokens, build projects, can export assets, agent commits changes, opening

My take: HTML-as-canvas is the most underappreciated idea in design tooling right now. The translation tax — the cost of converting Figma mockups into production code — consumes thousands of engineering hours per year. Eliminating that translation changes the economics of design-to-production. Paper and Huashu Design are early, but the direction is clear.

Design Tokens and Variables as Code

Design tokens are the connective tissue between design and code. A token is a named value: `color-primary`, `spacing-md`, `radius-sm`. In the old world, tokens lived in Figma's design system panel and were manually synced to CSS custom properties or JSON files. In the design-as-code world, tokens are variables in the same file as the design.



Design tokens defined as code variables in a JSON file, ready for agent consumption

Pencil variables system represents Variables. .pen hold values theme mode value, mode another maps to CSS proper

Workflow	Without Diffability	With Diffability
Design Review	Figma comments, screenshots in Slack	PR review with inline diff, approve/request changes
Design History	Figma's internal version history (proprietary)	<code>git log</code> , <code>git blame</code> , <code>revert</code> with <code>git revert</code>
Branching	Duplicate files or use Figma branches	<code>git checkout -b</code> , merge when ready
Automation	Manual export and validation	CI checks on tokens, linting, accessibility validation
Documentation	Agent can't modify binary files	Agent reads diff, writes changes, commits

Contrast this with Figma's binary `.fig` format. When a Figma file changes, `git diff` outputs "binary file changed." You can't see what changed. You can't review it line by line. You can't run automated checks. The file is opaque to every tool in your development pipeline except Figma itself.

Here's what a real `git diff` on a design file looks like. An agent changed a button component from a hardcoded red fill to a variable-bound primary color:

```
---
+++
@@ -1,2 +1,2 @@
- "red"
+ "primary"
```

Every detail is visible. The fill changed from hardcoded red to a variable reference. The corner radius changed from a magic number to a token. A reviewer can comment on this in a PR. An agent can generate this diff autonomously. A CI pipeline can validate that all fill values use variable references instead of hardcoded colors.

Consider the consequences for a team of five. With diffability, a color change requires: a meeting to discuss the change, a Figma file update, an export to the design team, manual QA across multiple touchpoints, visual QA pass through nothing was missed. Without diffability, the same change is a single variable in a `.pen` file, caught in a PR, reviewed and approved, and deployed through the same pipeline as code changes. It took two days to implement, 15 minutes.

Walk through the same task in both paradigms to feel the difference. The task: change the primary button color from blue to purple across a design system.

Walk through the same task in both paradigms to feel the difference. The task: change the primary button color from blue to purple across a design system.	Old world. Open Figma. Find the primary button component. Change the fill color. Check every instance where the component is used — some may have overrides. Update the design system documentation separately. Create a Figma branch or duplicate the file. Export screenshots for the developer. The developer receives the screenshots. They find every instance of the old blue in the CSS codebase. They change each one. They miss a few. The design and code drift apart.	New world. Open the <code>.pen</code> file. Change <code>"color-primary"</code> from <code>"#0066CC"</code> to <code>"#6B21A8"</code> . Commit. Push. The PR shows the exact diff — one line changed. Every component that references <code>\${color-primary}</code> updates automatically. The agent generates the corresponding CSS custom property change. CI validates that the new color passes WCAG contrast checks. The review is a single PR. The deployment happens in the same pipeline.	The convergence enabling this shift has three parts. First, <code>.pen</code> and <code>.op</code> files are code artifacts that agents can manipulate directly. Second, MCP connects agents to design tools as a standard protocol — no per-tool integration needed. Third, HTML serves as both the design medium and the production output, eliminating the translation tax.	This mental model — design files as code artifacts — underpins every workflow in the rest of the book. The tools change. The formats evolve. The principle stays the same: if your design file is a text-based artifact in your repo, every tool in your development pipeline can work with it.
--	---	---	--	---

Tip
Next: Chapter 05 puts this mental model into practice with Paper and Pencil — two connected canvas tools that make design-as-code tangible. You'll set up MCP servers, run real design-to-code workflows, and build a production website from a canvas design.

05 Paper and Pencil

*Connected
canvas tools,
MCP setup, and
design-to-code
workflows*

Two tools dominate the connected canvas space: Paper for HTML/CSS design and Pencil for vector design inside the IDE. This chapter walks through MCP server setup for each, real design-to-code workflows, and a complete pipeline from canvas design to production website. Every code block

is
copy-
paste
ready.

Paper:
Connected
HTML/CSS
Canvas for
Teams

Paper (paper.design) is a connected HTML/CSS canvas for teams shipping with AI agents. Built by the Paper team, it exposes a rich MCP server that lets agents read designs, export JSX, and write HTML back to the canvas. The key differentiator: design-to-code and code-to-design in the same loop.



Paper editor canvas with HTML/CSS design surface, layers panel, and connected tools

As of May 2026, designers at Vercel, Perplexity, Lovable, PostHog, Tailwind, Dub, Replicate, Zed, and Attio use Paper in production. The product lineup includes Paper Web (app.paper.design), Paper Desktop, Paper MCP, Paper Snapshot, and Paper Shaders.

Paper's design surface is HTML/CSS. The design artifact is HTML. The export is HTML. There is no translation layer between what you see on canvas and what ships in production. This makes Paper particularly effective for agent-driven workflows: an agent reads the HTML, modifies it, and writes it back. The canvas updates. No round-trip

Paper MCP server runs locally at `p://127.0.0.1:29979/mcp`. It starts automatically when Paper Kitop is open and a file is loaded. The setup differs by agent platform.

A terminal window titled "Terminal — zsh" with a macOS-style title bar (red, yellow, green buttons). It displays the following commands and output:

```
> claude mcp add paper --transport stdio npx @anthropic/paper-mcp-server Added MCP server paper to config
~/claude/mcp.json: { "mcpServers": { "paper": { "command": "npx", "args": ["@anthropic/paper-mcp-server",
"transport": "stdio" } } } > claude mcp list paper stdio npx @anthropic/paper-mcp-server Tools: get_selected
get_jsx, get_tokens, get_frame_tree, get_styles, get_assets, batch_get, export_nodes > claude "Verify Pa
connection" Connecting to Paper MCP server... ✓ Paper MCP server connected (v2.1.0) ✓ 8 tools registered
available ✓ Design files accessible: 4 projects, 23 frames > _
```

Paper MCP server setup showing configuration for Claude Code and OpenCode

Claude Desktop requires the `mcp-remote` package:

```
{
  "mcpServers": {
    "paper": {
      "command": "r
      "args": ["mcp
    }
  }
}
```

Cursor uses a plugin command:

```
$ /add-plu
```

After setup, verify the connection by asking your AI to create a simple element. Something like: "Create a red rectangle in Paper." If the rectangle appears on the canvas, the MCP server is working.

Warning

Warning: Paper Desktop is macOS and Windows only as of May 2026. Linux users cannot run the MCP server locally. The web version (app.paper.design) does not expose an MCP endpoint. This is the most significant limitation of the Paper workflow.

Warning

Warning: Paper Desktop is macOS and Windows only as of May 2026. Linux users cannot run the MCP server locally. The web version (app.paper.design) does not expose an MCP endpoint. This is the most significant limitation of the Paper workflow.

Paper's MCP server exposes 22 tools. Here are the most important ones for design-to-code workflows:

The full tool set includes `get_basic_info`,
`get_selection`, `get_node_info`,
`get_children`, `get_fill_image`,
`get_font_family_info`, `get_guide`,
`set_text_content`, `rename_nodes`,
`duplicate_nodes`, `move_nodes`, and
`finish_working_on_nodes`. Appendix B
covers every tool in detail.

The full tool set includes

```
get_basic_info ,  
get_selection ,  
get_node_info ,  
get_children ,  
get_fill_image ,  
get_font_family_info ,  
get_guide ,  
set_text_content ,  
rename_nodes ,  
duplicate_nodes ,  
move_nodes , and  
finish_working_on_nodes .
```

Appendix B covers every tool in detail.

Paper

Workflows:

Design-to-Code and Code-to-Design

Paper supports two primary workflows: design-to-code (export from canvas to code) and code-to-design (write code back to canvas). The MCP server makes both possible within a single agent session.



Workflow 1: Syncing tokens from Figma to Paper. Run both the Figma MCP server and the Paper MCP server in the same agent session. Here's the step-by-step process:

Workflow 2: Pulling real content from Notion into designs. Connect both Notion MCP and Paper MCP. Ask the agent to pull testimonials, articles, or data from Notion and populate a Paper design. This lets you create designs with real content — including content in different languages.

Pull the testimonials
the testimonials

The agent reads the Notion database, extracts the content, and uses `write_html` to populate the Paper cards. The result is a design filled with real copy instead of lorem ipsum.

Workflow 3: Building a website from a Paper design. This is the most powerful workflow. Select the hero frame in Paper. Ask the agent:

Pencil bridges design and development by putting both in the same environment. The key concepts:

Concept	What It Does	Design-as-Code Benefit
.pen files	Design artifacts stored in the repo	Diffable, versionable, reviewable
Variables	Design tokens with theme values	Tokens live in the design file
Components	Reusable design elements	Design system consistency
Slots	Component composition mechanism	Flexible component variants
Code on Canvas	Run code within the design surface	Design and code in one view
Design Libraries	Shared design system collections	Cross-project consistency

Chapter 04 covered the mental model behind these concepts. Pencil makes them tangible inside VS Code or Cursor. You open a `.pen` file in the editor. A canvas appears. You design on the canvas. The file saves to disk. Git tracks the changes. An agent reads the file via MCP and modifies it. The canvas updates.

The integration with the development workflow is tighter than Paper's. Pencil doesn't just connect to your IDE — it runs *inside* it. The canvas is a tab in your editor. You can see your code and your design side by side. When the agent modifies the `.pen` file, the canvas updates in real time, and the diff appears in your editor's SCM panel.

Compor work lik compon a compo instantia everyw you swa a compo same ca can hol images, compon update compon instance the sam React p operate canvas.

port capabilities round out the format. Support SVG, PNG, and Figma files. Export to SVG, PNG, PDF, and code generation. Support is useful for migrating existing designs from the `.fig` file in Pencil, and it converts styles, and components into the `.pen` file format. Conversion isn't perfect — some Figma-specific features like auto-layout translate differently. The format serves the structure well enough to support designing.

Design-to-Code Sync with Pencil

Pencil's design-to-code sync is bidirectional. Design changes propagate to code. Code changes propagate to design. The sync happens through the `.pen` file format and Pencil's MCP integration.

The sync workflow, with concrete agent prompts at each step:

1

Design in Pencil.

Open a `.pen` file in the IDE. Design on the canvas using Pencil's vector tools. Variables and components are defined in the file.

2

Agentic workflow file, e.g., the last

Real-time feedback

- 2 Connect the agent.** Ensure Paper MCP is configured in your agent (section 05.2). Open a terminal in the project folder. Start a new agent session. Verify the connection:

```
Create a small test rectangle in Paper to verify the MCP connection.
```

- 3 Generate the hero component** agent to build it:

```
Build a React + Tailwind
I have selected in Paper
to the CTA button.
```

The agent calls `get_screenshots` to capture the current design, `get_tailwind_markup` to extract the Tailwind markup, and `get_component_definition` to generate a `Hero.tsx` component with the necessary props and interactions.

S:

from Paper design"

Step	Tool	Agent Role
1. Design hero	Paper Desktop	None — human designs
2. Connect agent	Paper MCP	None — configuration
3. Generate hero	Paper MCP + Agent	Read design, generate code
4. Design remaining	Paper Desktop	None — human designs
5. Generate full page	Paper MCP + Agent	Read sections, compose page
6. Commit and deploy	Git + Agent	Commit, optionally deploy

The total time for this workflow: 30-90 minutes depending on design complexity. The agent handles the tedious parts — translating layouts, writing responsive breakpoints, mapping tokens to CSS custom properties. You focus on design direction. The agent handles execution.

Paper and Pencil solve overlapping problems from different angles. Here's how they compare:

Dimension	Paper
Design surface	HTML/CSS canvas
File format	HTML (in Paper's cloud)
Agent connection	MCP server (HTTP)
Code export	<code>get_jsx</code> → Tailwind/JSX
Best for	Web designs, landing pages
Linux support	No (Desktop required)

06 Open Design and OpenPencil

Open-source design frameworks, 31 skills, and agent-powered canvas

Open Design (43k+ stars) and OpenPencil (3k+ stars) are the open-source alternatives to the commercial tools covered in chapter 05. Open Design provides 31 composable skills and 129 design systems that run on any of 16 agent CLIs. OpenPencil is the first open-source AI-native vector design tool with concurrent Agent Teams. This chapter walks

through setup, skill selection, and real workflows for both.	Open Design: The Open- Source Claude Design Alternative	Open Design is a local-first, open-source alternative to Anthropic's Claude Design. As of May 2026, it has 43.6k GitHub stars, 5k forks, and an Apache-2.0 license. It does not ship its own agent. Instead, it wires existing coding agents into a skill-driven design workflow.	The context: Anthropic released Claude Design on April 17, 2026 with the Opus 4.7 model. It went viral. It also stayed closed-source, paid-only, cloud-only, and locked to Anthropic's model. Open Design provides the same artifact-first loop — prompt, generate, preview, iterate — without any of the lock-in.	The architecture is web-plus-daemon. A local daemon scans your PATH for 16 agent CLIs on startup. When you prompt Open Design, the daemon spawns your chosen CLI in the project folder. The agent gets real <code>Read</code> , <code>Write</code> , <code>Bash</code> , and <code>WebFetch</code> tools against the real on-disk environment. Every generated artifact renders in a sandboxed srcdoc iframe.	This architecture has important implications. Because the daemon spawns a real CLI agent — not a simulated one — the agent has full access to the file system, can install packages, read existing code, and run build commands. The artifacts it generates are real HTML files that live on disk, not just rendered previews. You can take the
--	--	---	--	---	---

output The daemon also handles security. Internal-IP and SSRF requests are blocked at the
and edge, preventing the agent from accessing your local network services. Loopback
ship it access is allowed for Ollama and LM Studio, so you can run local models without
directly. exposing them to the internet. The sandboxed iframe prevents generated artifacts from
 executing arbitrary

JavaScript in your browser context.	Open Design's open-source dependencies are worth understanding because they shape its capabilities. The project builds on four key repos: Huashu Design provides the design-philosophy compass (the Junior Designer workflow, brand-asset protocol, and anti-AI-slop rules). Guizang PPT contributes the deck mode. Open Cowork AI provides the UX patterns (streaming artifact loop, sandboxed iframe preview, live agent panel). Multica AI provides the daemon-and-runtime architecture (PATH-scan agent detection, local daemon, agent-as-teammate pattern). Each dependency is bundled with its original license.	The supported agent CLIs span the full ecosystem:
-------------------------------------	--	---

Sixteen total as of May 2026. The daemon detects whichever ones are installed and offers them in the model picker. No need to install all 16 — install the agent you prefer, and Open Design detects it automatically.

istence	Open Design can	The live preview updates as the agent works. When	The daemon manages the full
sqlite	also import Claude	the daemon spawns the agent CLI, it streams the	lifecycle: starting the agent,
ects,	Design export ZIPs	agent's output in real time. You see the artifact	routing its output to the
ns,	via POST	building section by section — first the layout	preview iframe, capturing the
tabs,	/api/import/claude-	skeleton, then the content, then the styling. A live	generated files to disk, and
	design . If you	todo card tracks the agent's progress, showing	cleaning up the agent
	started with Claude	which tasks are in_progress and which are	process when the session
is local.	Design and want to	completed . This streaming feedback loop is more	ends. You can run multiple
ves	migrate, the import	informative than waiting for a final result, because	sessions concurrently —
ne	handles the	you can spot problems early and redirect the agent	different agents working on
	conversion.	before it completes the wrong output.	different artifacts in the same
a cloud			project.

kills are composable task definitions. There are 31 as of May 2026, organized by scenario:

Skills follow the SKILL.md convention covered in chapter 03, with extended `od:` frontmatter for Open Design-specific metadata. The daemon selects skills automatically based on your prompt. Say "landing page" and it picks `saas-landing`. Say "pitch deck" and it picks `guizang-ppt` or `simple-deck`. You can also force a specific skill by name:

Color

Prim

- Prime

- Prime

Neut

- Backg

- Foreg

- Muted

Type

- Font

- Headi

- Body:

Spac

- Unit:

- Scale

Switch a design system and the next render uses the new tokens. No manual color palette changes. No font swaps. The design system drives everything.

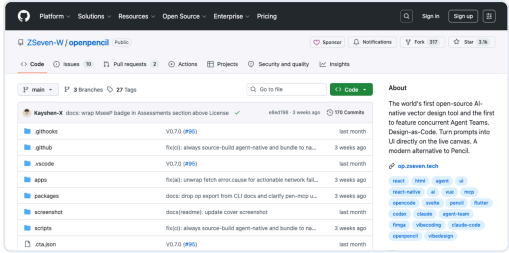
Visual directions are curated aesthetic philosophies. Five schools as of May 2026: Editorial Monocle, Modern Minimal, Warm Soft, Tech Utility, and Brutalist Experimental. Each ships a deterministic OKLch palette and font stack. The direction picker presents these as radio options — one click sets the entire visual language.

My take: The skill system is Open Design's killer feature. 31 skills means 31 starting points for different design tasks. Compare this to Claude Design, which has one generic artifact generation mode. The skill selection is not perfect — sometimes the daemon picks the wrong skill for ambiguous prompts — but you can always override it. The 129 design systems are impressive on paper, but most teams will use 3-5. The value is in having the right system available, not the quantity.

ne question form
sign's smartest
st agent design
ne from vague
locking the brief
eration is a
ern that
y improves
ity. I've tested
20+ runs:
puts are
y better than
ones.

**OpenPencil:
AI-Native
Vector
Design with
Agent
Teams**

OpenPencil is the first open-source AI-native vector design tool. As of May 2026, it has 3.1k GitHub stars, 317 forks, and an MIT license. The differentiator: concurrent Agent Teams — multiple AI agents working on different sections of a design simultaneously.



OpenPencil canvas with concurrent Agent Teams working on different design sections

The architecture
app plus native d
(macOS, Window
via Electron). The
rendering engine
CanvasKit/Skia v
WASM with GPU
acceleration. The
stack: React 19, T
Start, Tailwind CS
shadcn/ui, Zustand
Nitro, Electron 35
and Vite 7.

see on the canvas: a document containing canvases, canvases
aining text and shapes. Each property is a readable key-value pair. A
actly what changed — a color, a font size, a layout direction.

Setting Up OpenPencil and Its MCP Server

OpenPencil has three
installation paths:
package manager, CLI, or
Docker.

OpenCode

(`opencode.json`):

```
{
  "mcp": {
    "openpencil": {
      "type": "stdio",
      "command": "npx",
      "args": ["pen-mcp"],
      "enabled": true
    }
  }
}
```

Codex CLI: Settings >

MCP Servers > enter
`npx pen-mcp` as the
command.

The MCP server provides style guide tools (`get_style_guide`) and an incremental codegen pipeline (`codegen_plan` , `codegen_submit` , `codegen_clean`). The codegen pipeline streams output incrementally, once, which improves reliability for large exports.

Concurrent Agent Teams: Parallel Design on Canvas

OpenPencil's signature feature is concurrent Agent Teams. An orchestrator decomposes a complex page into spatial sub-tasks. Multiple AI agents then work on different sections simultaneously — one on the hero, another on features, a third on the footer.



Same component generated by Open Design (left) and OpenPencil (right) showing output differences

The layered workflow runs in three phases:

- 1
- design_skeleton** — The orchestrator creates the page layout grid, section boundaries, spacing rules. This is the architectural phase. The orchestrator decides how many sections the page needs, their relative sizes, and the layout strategy.

```
# Orchestrator prompt (simplified)
"Analyze the design brief and create a skeleton
- Determine section count and order
- Set section heights and spacing
- Define the grid system
Output: a .op file with empty frames for each section"
```

2 design_content — Agent Teams fill in each section in parallel. Each agent gets a spatial sub-task. Here's a concrete example for a SaaS landing page:

3

```
# Agent 1: Hero section
"Design the hero section in the 'hero' frame. Include:
headline, subhead, CTA button, and hero image."

# Agent 2: Features section
"Design the features section in the 'features' frame.
Show 3 feature cards in a grid layout."

# Agent 3: Footer section
"Design the footer section in the 'footer' frame.
Include links, social icons, and copyright."
```

Streaming indicators show per-member progress on the canvas. Each section fills in independently. The agents do not wait for each other.

The entire three-phase flow for a typical landing page takes 2-5 minutes with Claude Sonnet as the agent. The skeleton phase runs in under 30 seconds. The parallel content phase takes 1-3 minutes depending on section complexity. The refine pass takes 30-60 seconds.

Tier	Models	Adaptation
Full-tier	Claude Opus/Sonnet	Complete prompts with extended thinking
Standard-tier	GPT-4o, Gemini Pro, DeepSeek	Full prompts, thinking disabled
Basic-tier	MiniMax, Qwen, Llama, Mistral	Simplified nested-JSON prompts

Segmented prompt retrieval loads only the design knowledge needed for each sub-task. A hero section agent gets hero-relevant tokens, not the full design system. This keeps context windows efficient and reduces hallucination from information overload.

OpenPencil also ships an embeddable SDK for building custom design tools. The monorepo packages include `pen-types`, `pen-core`, `pen-engine` (headless), `pen-react` (React UI SDK), `pen-codegen`, `pen-figma`, `pen-renderer`, `pen-mcp`, `pen-sdk`, and `pen-ai-skills`. Chapter 11 covers multi-agent patterns in depth.

Comparing Open Design and OpenPencil

The recommendation depends on your use case. Open Design is better for generating complete artifacts from prompts — landing pages, pitch decks, dashboards, marketing materials. OpenPencil is better for interactive vector design work where you need a canvas, component system, and the option to run multiple agents in parallel.

The recommendation depends on your use case. Open Design is better for generating complete artifacts from prompts — landing pages, pitch decks, dashboards, marketing materials. OpenPencil is better for interactive vector design work where you need a canvas, component system, and the option to run multiple agents in parallel.

If you need to generate a quick prototype or marketing page, Open Design's skill system and 129 design systems give you more starting points. If you're building a production component library with agent assistance, OpenPencil's vector canvas and concurrent Agent Teams are the stronger choice.

My take: Open Design has the larger community and more breadth. OpenPencil has the more interesting technical architecture. The concurrent Agent Teams feature is genuinely novel — no other design tool does parallel multi-agent design on a shared canvas as of May 2026. But it's newer and less battle-tested. For production work today, Open Design is the safer choice. For exploring where agentic design is heading, OpenPencil is worth watching closely.

Tip

Next: Chapter 07 covers Huashu Design (14k+ stars) — the HTML-native design skill that turns a single sentence into production-grade prototypes, slide decks, and motion design. It takes the HTML-as-canvas concept from chapter 04 and pushes it further than any other tool in the ecosystem.

07 Huashu Design

Huashu Design (14.1k GitHub stars as of May 2026) is the most opinionated design skill in the agent ecosystem. It treats HTML as the design surface, not the export target. This chapter walks through installing it, understanding its workflow, and producing real artifacts - -- prototypes, slide decks, motion design --- with an AI agent as your junior designer.

What Makes Huashu Design Different

Most design tools give you a canvas, panels, and property inspectors. Huashu Design gives you none of that. The entire tool is a `SKILL.md` file --- 811 lines of instructions that teach any skill-capable agent how to produce production-grade visual artifacts using nothing but HTML, CSS, and JavaScript.

The distinction matters. When I say "HTML-native design," I don't mean "export to HTML." I mean the agent embodies different roles depending on the task - -- UX designer for prototypes, animator for motion design, slide designer for decks --- and produces HTML as the primary artifact. There is no graphics tool layer. The conversation *is* the interface.

Created by 花叔 (Huasheng, @AlchainHust), an independent developer whose other work includes *A Book on DeepSeek* and the Nüwa.skill framework (12k+ stars). As of May 2026, Huashu Design is MIT licensed, free for personal and commercial use, and runs on Claude Code, Cursor, Codex, Trae, Hermes, OpenClaw --- any agent that reads markdown skill files.

Attribute	Detail
GitHub stars	14.1k (May 2026)
License	MIT
Author	花叔 (@AlchainHust)
Format	SKILL.md (markdown skill file)
Agent compatibility	Any skill-capable agent
Output formats	HTML, MP4, GIF, PPTX, PDF
Install	<code>npx skills add alchaincyf/huashu-design</code>

```
$ npx
```

That drops the skill into your agent's skill directory. No app to install. No plugin to configure. No Figma integration to set up. The agent reads the skill file and follows its instructions.

The repository structure tells you everything about what the skill can do:

```
huashu-design/  
├─ SKILL.md  
├─ assets/  
│   ├── animations.jsx  
│   ├── ios_frame.jsx  
│   ├── deck_stage.js  
│   ├── design_canvas.jsx  
│   └─ showcases/  
├─ references/  
│   ├── animation-pitfalls.md  
│   ├── design-styles.md  
│   ├── slide-decks.md  
│   ├── editable-pptx.md  
│   ├── critique-guide.md  
│   └─ video-export.md  
├─ scripts/  
│   ├── render-video.js  
│   ├── convert-formats.sh  
│   ├── add-music.sh  
│   ├── export_deck_pdf.mjs  
│   ├── export_deck_pptx.mjs  
│   └─ html2pptx.js  
└─ demos/
```

The `SKILL.md` is the main instruction file. The `assets/` directory holds starter components --- animation APIs, iOS device frames, deck templates. The `references/` directory contains drill-down docs the agent can load when it needs deeper guidance on a specific task. The `scripts/` directory is the export toolchain.

After installation you start a conversation with your agent and describe what you want. The skill activates when the agent detects a design task. You don't invoke it explicitly.

Eyeball with Playwright before delivery

"Fixing a misunderstanding early is 100x cheaper than fixing it late." This isn't a platitude. It's the engineering principle behind the entire workflow. The agent writes `/* assumption: user wants dark mode */` directly into the HTML. You see the assumption. You correct it immediately. The alternative --- the agent spending 10 minutes building a polished design on wrong assumptions --- wastes everyone's time.

The skill uses the React + Babel pattern for live rendering. All starter components load via `<script type="text/babel">` tags, which means the agent writes JSX that compiles in the browser. No build step. No bundler. The HTML file is the deliverable.

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://unpkg.com/react@18/umd/react.production.min.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.production.min.js"></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    function App() {
      return <div style={{padding: 32}}>
        /* placeholder: hero section */
        <div style={{
          height: 400,
          background: 'var(--surface-muted)',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          borderRadius: 12
        }}>
          [Hero placeholder – awaiting copy]
        </div>
      </div>;
    }
    ReactDOM.createRoot(document.getElementById('root')).render(<App/>);
  </script>
</body>
</html>
```

The Tweaks system enables live variant switching. The agent renders multiple design variations with parameter controls. You adjust parameters and see changes instantly, without regenerating the entire file.

My take: The Junior Designer Workflow is the most important contribution Huashu Design makes to the agent design ecosystem. Most agent-generated design fails because the agent tries to produce a final artifact immediately. Huashu's approach --- hypothesis first, polish later --- maps onto how real designers actually work. I use this pattern even when I'm not using Huashu Design specifically.

Core Asset Protocol: Brand-Consistent Output

The hardest rule in the skill. When your task involves a specific brand, the agent must follow a 5-step process before writing a single line of HTML.

The problem the protocol solves: agents default to generic visuals. Without real brand assets, an iPhone prototype looks like every other phone. A Volta Motors slide deck looks like a generic car pitch. The Core Asset Protocol forces the agent to gather real assets first.

- 1 **Ask** for 6 asset types: logo, product shots, UI screenshots, color palette, fonts, brand guidelines. The agent presents a checklist and waits.

- 2 **Search official channels:** `<brand>.com/brand` , `<brand>.com/press` , `brand.<brand>.com` , product pages, launch films. Three fallback paths per asset type.

- 3 **Download by asset type.** Logo: SVG file, then inline SVG extraction from homepage, then social media avatar. Product shots: product page hero, then press kit, then YouTube video frame, then AI-generated from reference. UI: App Store screenshots, then official video frames, then user screenshots.

- 4 **Verify and extract.** Check resolution, fidelity, freshness. Grep exact colors from real assets.

